

PHOENIX

Algorithm Specifications And Supporting Documentation

Zhongxiang Zheng, Chunhuan Zhao, Anyu Wang, Guangwu Xu,
Zhichen Yan, Yingxue Ma, Jingtao Wang, Wanshan Xu, Zhengtao
Jiang, Congming Wei, Huayun Tang, Chen Jia

Table of Contents

1	Introduction	1
1.1	Design Rationale	1
1.2	Performance and Parameterization	2
1.3	Feature Statement	2
2	Notation	3
2.1	Data Types and Operators	3
2.2	Basic Definitions of Security	4
2.3	Hash Functions and Pseudorandom Functions	7
2.4	Address and Member Functions	8
3	Building Blocks 1: GWOTS+C	8
3.1	The Design Rationale of GWOTS+C	9
3.2	Parameter Selection Strategy	11
3.3	The Algorithmic Specification	12
3.4	Correctness and Security Analysis	16
4	Building Blocks 2: Transposed FORS(TFORs)	17
4.1	Design Rationale	17
4.2	The Octopus Algorithm and Inverse Octopus Algorithm	18
4.3	The TFORS Scheme	19
4.4	Security Analysis	24
5	Building Block 3: XMSS and Hypertree	26
5.1	XMSS Layer	27
5.2	Hypertree	29
6	Phoenix Scheme	30
6.1	Key Generation	30
6.2	Signature Generation	31
6.3	Signature Verification	33
7	Security Proof of Phoenix	33
8	Parameter Sets for Phoenix	37
8.1	Parameter Notation	37
8.2	Concrete Parameter Choices	37
9	Implementation and Performance	39
9.1	Experimental Setup	39
9.2	Advantages and Limitations of Phoenix	40
A	Address and Member functions	43
B	Byte-String and Integer Conversions	46
C	Comparison with SPHINCS+ and SPHINCS+C	47

1 Introduction

Stateless hash-based signatures are of particular interest in the post-quantum setting because they rely on conservative hash-based assumptions and avoid the state-management requirements of XMSS-type systems [6, 12]. More broadly, hash-based signature research has developed along several major routes, including Lamport-type one-time signatures [11], Merkle-tree signatures [13], and stateless hypertree systems such as SPHINCS⁺/SLH-DSA [4, 7]. The latter category also includes optimization-oriented variants built on the same line of research, such as SPHINCS⁺C [10] and SPHINCS⁺ α [16], which typically employ few-time signature primitives at the bottom layer rather than strict one-time ones. Although these approaches differ in key management and system architecture, they share a common practical bottleneck: the composition of one-time or few-time primitives with tree-based authentication leads to large signature sizes, substantial hashing costs, and limited flexibility in balancing compression against signing efficiency.

1.1 Design Rationale

The primary design objective of our proposed algorithm is to reduce signature overhead at the lower layers of a stateless hash-based signature system without altering the architectural advantages of the upper layers. In particular, the design preserves the stateless hypertree framework and the domain-separated hash-based interface, while re-balancing the size-speed tradeoff inside the two signature components that dominate overall signature size, namely the one-time and few-time primitives.

To overcome the bottlenecks of the existing designs, we introduce a new signature scheme called Phoenix, which is built on two novel primitives: GWOTS+C, a generalized WOTS+C one-time signature scheme, and TFORS, a transposed FORS few-time signature scheme.

1. The one-time signature component is realized by GWOTS+C, a generalized WOTS+C design that interpolates between WOTS⁺ [9] and fixed-point WOTS+C [10]. Its defining mechanism is the replacement of a single checksum target by an authenticated checksum-offset window: instead of forcing Winternitz checksum to one exact value, the signer accepts a window $[L, U]$ and authenticates only the exact offset $C - L$. This preserves the benefit of omitting selected chains while avoiding the high rejection cost associated with the fixed-point design, which requires forcing the checksum to a single exact value.
2. The few-time signature component is realized by TFORS, which transposes the organization of FORS secret-value sets across Merkle trees. This restructuring changes where overlap occurs in the authentication structure, thereby

making Octopus-based pruning and forced pruning substantially more effective. In operational terms, TFORS converts structural overlap into a directly exploitable source of compression.

The two components are integrated into a hypertree framework with both fast-oriented and size-oriented parameter families.

1.2 Performance and Parameterization

Following the design principle of SPHINCS⁺ [4], Phoenix provides two types of parameter sets for each security level: size-optimized (suffix “s”) and speed-optimized (suffix “f”). All parameter sets maintain the same security level and support up to 2^{64} signatures, matching the guarantees of the corresponding SPHINCS⁺ NIST submission variants.

Table 1 summarizes the signature size comparison for 128, 192, and 256-bit security levels. For 128-bit security, Phoenix-SHAKE-128s achieves 6,252 bytes versus SPHINCS⁺-128s’s 7,856 bytes, a reduction of 20%. Phoenix-SHAKE-128f achieves 13,668 bytes versus 17,088 bytes, also a 20% reduction. Similar reductions are observed at higher security levels.

Table 1: Signature size comparison: SPHINCS⁺ vs. Phoenix (bytes)

Security	Small (s)		Fast (f)	
	SPHINCS ⁺	Phoenix	SPHINCS ⁺	Phoenix
128-bit	7,856	6,252 (-20%)	17,088	13,668 (-20%)
192-bit	16,224	13,290 (-18%)	35,664	30,880 (-13%)
256-bit	29,792	26,624 (-11%)	49,856	47,464 (-5%)

The reduction in signature size comes with moderate computational overhead. As shown in Section ??, Phoenix achieves these size reductions at the cost of a measurable increase in signing and verification times. Table 4 lists the concrete parameters for all Phoenix variants.

In summary, Phoenix achieves smaller signatures than SPHINCS⁺ at equivalent security levels, making it particularly attractive for bandwidth-constrained applications such as blockchain transactions and code signing.

1.3 Feature Statement

Phoenix offers the following combination of features that distinguish it from prior stateless hash-based signature schemes:

- **Reduced signature size.** By combining a generalized Winternitz checksum with transposed FORS pruning, Phoenix achieves 5–20% smaller signatures than SPHINCS⁺ at equivalent security levels (see Table 1), making it particularly attractive for bandwidth-constrained applications such as blockchain transactions and code signing.
- **Stateless operation.** The scheme inherits the fully stateless hypertree design of SPHINCS⁺, requiring no state updates across signatures and supporting up to 2^{64} signing operations per key pair.
- **Conservative security.** All constructions are proved secure under standard post-quantum assumptions (one-wayness, second-preimage resistance, and pseudorandomness of the underlying hash functions) in the random oracle model, with explicit security bounds that incorporate interleaved target subset resilience (ITSR) for the few-time component.
- **Flexible parameterization.** Two families of parameter sets are provided for each security level: size-optimized (suffix “s”) and speed-optimized (suffix “f”), enabling practitioners to select the trade-off that best matches their system’s requirements. The parameter space additionally supports mixed-width Winternitz profiles that allow fine-grained control over the time–space trade-off within the one-time signature layer.
- **Transparent integration.** Phoenix can be instantiated using the same symmetric primitives (SHA-256, SHAKE256, etc.) and domain-separation framework as SLH-DSA, facilitating comparison, implementation reuse, and potential migration paths.

2 Notation

This section fixes the notation, data representations, and conversion routines used in the rest of the paper.

2.1 Data Types and Operators

Data types and Bytes. Let $\mathbb{B} = \{0, \dots, 255\}$ denote the set of bytes. For an integer $k \geq 0$, we write $\mathbb{B}^k$ for the set of byte strings of length k , and $\mathbb{B}^*$ for the set of finite byte strings of arbitrary length. For a byte string S , we use zero-based indexing, where $S[i]$ denotes the i -th byte, and $S[i : j]$ denotes the substring starting at index i and ending at $j - 1$.

Operators and Conversions. We use the following operators and conversion routines throughout the algorithms:

- $y \leftarrow x$ denotes assignment of the value x to the variable y .
- If S is a finite set, then $y \leftarrow_R S$ denotes sampling y uniformly at random from S .

- For byte strings X and Y , $X \parallel Y$ denotes concatenation. If X and Y have the same length, $X \oplus Y$ denotes their byte-wise XOR.
- The symbols $\ll$ and $\gg$ denote logical left and right shifts, respectively. We use $\&$ for bitwise AND and $\%$ for the modulo operation.
- All integer–byte-string conversions use big-endian byte order.
- **toByte** (x, n) converts $x \bmod 256^n$ to an n -byte big-endian string.
- **toInt** (X, n) maps an n -byte string X to the corresponding non-negative integer under big-endian interpretation.

Detailed conversion algorithms and fixed-width digest parsing conventions are given in Appendix B.

Table 2: Glossary of Symbols and Parameters

Symbol	Description
n	Byte length of public/private keys and hash digests. $8n$ represents the security parameter.
h	The height of the hypertree.
d	The number of layers in the hypertree.
h'	The height of each individual XMSS tree $h' = h/d$.
m	The number of message chains in General WOTS+C.
d_{off}	The number of checksum-offset chains in General WOTS+C.
ℓ	The total number of chains in General WOTS+C, $\ell = m + d_{\text{off}}$.
k	The number of secret values revealed in a TFORS signature ($k \leq k'$).
k'	The number of secret value sets in TFORS.
t	The number of secret values per set in TFORS, satisfying $t = 2^a$.
a	$\log_2$ of the number of secret values per TFORS set.
h_{TFORS}	Height of the TFORS Merkle tree, $h_{\text{TFORS}} = a + \log_2 k'$.
ℓ_{msg}	Length of the message digest, $\ell_{\text{msg}} = L_{\text{TFORS}} + \lceil (h - h')/8 \rceil + \lceil h'/8 \rceil$, where $L_{\text{TFORS}} = \max(n, \lceil k \cdot a/8 \rceil)$.
addr	A 32-byte structure used to separate hash function domains.

2.2 Basic Definitions of Security

We recall the security notions used in the analysis. For a primitive $\mathbf{F}$ and an adversary $\mathcal{A}$, we write $\text{Adv}_{\mathbf{F}}^{\text{PROP}}(\mathcal{A})$ for the advantage of $\mathcal{A}$ against property PROP.

Definition 1 (EU-CMA). Let $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Verify})$ be a digital signature scheme. The EU-CMA security is defined via the following experiment between a challenger and an adversary $\mathcal{A}$:

1. The challenger runs $(\text{SK}, \text{PK}) \leftarrow \text{KeyGen}(1^{\lambda=8n})$ and gives PK to $\mathcal{A}$.
2. $\mathcal{A}$ adaptively queries a signing oracle $\mathcal{O}_{\text{Sign}}(\text{SK}, \cdot)$ with messages M_i . The oracle returns $\sigma_i \leftarrow \text{Sign}(\text{SK}, M_i)$. Let $\mathcal{Q}$ be the set of queried messages.
3. $\mathcal{A}$ outputs a forgery (M^*, σ^*) .
4. $\mathcal{A}$ wins if $\text{Verify}(\text{PK}, M^*, \sigma^*) = 1$ and $M^* \notin \mathcal{Q}$.

The EU-CMA advantage of $\mathcal{A}$ against Σ is

$$\text{Adv}_{\Sigma}^{\text{EU-CMA}}(\mathcal{A}) = \Pr[\mathcal{A} \text{ wins}].$$

Let $\mathbf{F} : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{N}$ be a keyed, or tweakable, hash-function family, where $\mathcal{K}$ is the key or tweak space, $\mathcal{M}$ is the input space, and $\mathcal{N}$ is the output space.

Definition 2 (One-Wayness). A hash function family is one-way (preimage resistant) if it is computationally infeasible to find the input given the output. The advantage of an adversary $\mathcal{A}$ is:

$$\text{Adv}_{\mathbf{F}}^{\text{OW}}(\mathcal{A}) = \Pr[k \leftarrow_R \mathcal{K}; x \leftarrow_R \mathcal{M}; y \leftarrow \mathbf{F}(k, x); x' \leftarrow \mathcal{A}(k, y) : \mathbf{F}(k, x') = y].$$

Definition 3 (Second-Preimage Resistance). A hash function family is second-preimage resistant if, given a randomly chosen key k and input x , it is hard to find a different input that hashes to the same output. The advantage is:

$$\text{Adv}_{\mathbf{F}}^{\text{SPR}}(\mathcal{A}) = \Pr[k \leftarrow_R \mathcal{K}; x \leftarrow_R \mathcal{M}; x' \leftarrow \mathcal{A}(k, x) : x' \neq x \wedge \mathbf{F}(k, x') = \mathbf{F}(k, x)].$$

Definition 4 (Collision Resistance). A hash function family is collision-resistant if it is hard to find any two distinct inputs that map to the same output. The advantage is:

$$\text{Adv}_{\mathbf{F}}^{\text{CR}}(\mathcal{A}) = \Pr[k \leftarrow_R \mathcal{K}; (x, x') \leftarrow \mathcal{A}(k) : x' \neq x \wedge \mathbf{F}(k, x') = \mathbf{F}(k, x)].$$

For tweakable hash functions, we also use target collision resistance. In this notion, the challenger fixes a target input, and the adversary must find a distinct input that collides with the target under the same key or tweak.

Definition 5 (Undetectability). Undetectability requires that the output of $\mathbf{F}$ on a uniformly sampled input be computationally indistinguishable from a uniformly sampled element of the output space. The distinguishing advantage of $\mathcal{A}$ is

$$\text{Adv}_{\mathbf{F}}^{\text{UD}}(\mathcal{A}) = |\Pr[\mathcal{A}(k, \mathbf{F}(k, x)) = 1] - \Pr[\mathcal{A}(k, u) = 1]|,$$

where $k \leftarrow_R \mathcal{K}$, $x \leftarrow_R \mathcal{M}$, and $u \leftarrow_R \mathcal{N}$.

This notion was used by Dods et al. [8] in the analysis of WOTS.

Definition 6 (Pseudorandom Function). A keyed function $\text{PRF} : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{N}$ is a secure pseudorandom function if its outputs are computationally

indistinguishable from those of a truly random function $\mathcal{R} : \mathcal{M} \rightarrow \mathcal{N}$. The advantage of a distinguisher $\mathcal{A}$ given oracle access is:

$$\mathbf{Adv}_{\mathbf{PRF}}^{\mathbf{PRF}}(\mathcal{A}) = \left| \Pr \left[\mathcal{A}^{\mathbf{PRF}(k, \cdot)}() = 1 \right] - \Pr \left[\mathcal{A}^{\mathcal{R}(\cdot)}() = 1 \right] \right|,$$

where $k \leftarrow_R \mathcal{K}$.

Definition 7 (Tweakable Hash Function [4]). Let $n, \alpha \in \mathbb{N}$, $\mathcal{P}$ the public parameters space and $\mathcal{T}$ the tweak space. A tweakable hash function is an efficient function

$$\mathcal{H} : \mathcal{P} \times \mathcal{T} \times \{0, 1\}^\alpha \rightarrow \{0, 1\}^n,$$

mapping an α -bit message M to an n -bit hash value using a public parameter $P \in \mathcal{P}$ and a tweak $T \in \mathcal{T}$. We sometimes write $\mathcal{H}_{P,T}(M)$ in place of $\mathcal{H}(P, T, M)$. The hash functions $\mathbf{F}$, $\mathbf{H}$, and $\mathbf{T}_\ell$ used in the security analysis are all instances of tweakable hash functions.

Definition 8 (Single-Message Target Collision Resistance, SM-TCR [4]). Let $\mathcal{H}$ be a tweakable hash function. We define the success probability of any adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ against the SM-TCR security of $\mathcal{H}$. The definition is parameterized by the number of targets p for which it must hold that $p \leq |\mathcal{T}|$. In the definition, $\mathcal{A}_1$ is allowed to make p queries to an oracle $\mathcal{H}(P, \cdot, \cdot)$. We denote the set of $\mathcal{A}_1$'s queries by $Q = \{(T_i, M_i)\}_{i=1}^p$ and define the predicate $\text{DIST}(\{T_i\}_{i=1}^p) = (\forall i, k \in [1, p], i \neq k) : T_i \neq T_k$, i.e., DIST outputs 1 iff all tweaks are distinct. The success probability is:

$$\mathbf{Succ}_{\mathcal{H}, p}^{\text{SM-TCR}}(\mathcal{A}) = \Pr \left[\begin{array}{l} P \leftarrow_R \mathcal{P}; \\ S \leftarrow \mathcal{A}_1^{\mathcal{H}(P, \cdot, \cdot)}(); \\ (j, M) \leftarrow \mathcal{A}_2(Q, S, P) : \\ \mathcal{H}(P, T_j, M_j) = \mathcal{H}(P, T_j, M) \\ \wedge M \neq M_j \wedge \text{DIST}(\{T_i\}_{i=1}^p) \end{array} \right].$$

We define the insecurity against p -target, time- ξ PQ-SM-TCR adversaries as the maximum success probability of any possibly quantum adversary $\mathcal{A}$ with p targets and running time $\leq \xi$:

$$\text{InSec}^{\text{PQ-SM-TCR}}(\mathcal{H}; \xi, p) = \max_{\mathcal{A}} \left\{ \mathbf{Succ}_{\mathcal{H}, p}^{\text{SM-TCR}}(\mathcal{A}) \right\}.$$

Definition 9 (Single-Message Decisional Second-Preimage Resistance, SM-DSPR [4]). Let $\mathcal{H}$ be a tweakable hash function. We define the advantage of any adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ against the SM-DSPR security of $\mathcal{H}$. The definition is parameterized by the number of targets p for which it must hold that $p \leq |\mathcal{T}|$. In the definition, $\mathcal{A}_1$ is allowed to make p queries to an oracle $\mathcal{H}(P, \cdot, \cdot)$. We denote the set of $\mathcal{A}_1$'s queries by $Q = \{(T_i, M_i)\}_{i=1}^p$ and require $\text{DIST}(\{T_i\}_{i=1}^p) = 1$. Let $\mathcal{S}_{Q,P} = \{i \in [1, p] : \exists M \neq M_i \text{ s.t. } \mathcal{H}(P, T_i, M) = \mathcal{H}(P, T_i, M_i)\}$ be the set of queries that have a distinct second preimage under P . We define a bit b and

a set $\mathcal{S}'$ as follows: with probability $1/2$, $b = 0$ and $\mathcal{S}' = \mathcal{S}_{Q,P}$; with probability $1/2$, $b = 1$ and $\mathcal{S}'$ is a random subset of $[1, p]$ with the same cardinality as $\mathcal{S}_{Q,P}$. The adversary's advantage is:

$$\mathbf{Adv}_{\mathcal{H},p}^{\text{SM-DSPR}}(\mathcal{A}) = \left| \Pr \left[\begin{array}{l} P \leftarrow_R \mathcal{P}; \\ S \leftarrow \mathcal{A}_1^{\mathcal{H}(P, \cdot, \cdot)}(); \\ b \leftarrow_R \{0, 1\}; \\ \mathcal{S}' \text{ as above}; \\ b' \leftarrow \mathcal{A}_2(Q, S, P, \mathcal{S}') : b' = b \end{array} \right] - \frac{1}{2} \right|.$$

We define the insecurity against p -target, time- ξ PQ-SM-DSPR adversaries as:

$$\text{InSec}^{\text{PQ-SM-DSPR}}(\mathcal{H}; \xi, p) = \max_{\mathcal{A}} \left\{ \mathbf{Adv}_{\mathcal{H},p}^{\text{SM-DSPR}}(\mathcal{A}) \right\}.$$

2.3 Hash Functions and Pseudorandom Functions

We define the following domain-separated hash-based functions: $\mathbf{PRF}_{msg}$, $\mathbf{H}_{msg}$, $\mathbf{PRF}$, $\mathbf{T}_\ell$, $\mathbf{H}$, $\mathbf{F}$, and $\mathbf{H}_2$.

- $\mathbf{PRF}_{msg}(\mathbf{SK.prf}, opt_rand, M) : \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^* \rightarrow \mathbb{B}^n$ is a pseudorandom function that generates the randomizer R for the message to be signed.
- $\mathbf{H}_{msg}(R, \mathbf{PK.seed}, \mathbf{PK.root}, M) : \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^* \rightarrow \mathbb{B}^{\ell_{msg}}$ computes the message digest used by the signing algorithm, where $\ell_{msg} = L_{\text{TFORS}} + \lceil (h - h')/8 \rceil + \lceil h'/8 \rceil$. Here, $L_{\text{TFORS}} = \max(n, \lceil k \cdot a/8 \rceil)$ is the length required to select k secret value sets from k' sets in TFORS, $\lceil (h - h')/8 \rceil$ is the length required to select the tree index idx_{tree} in the hypertree, and $\lceil h'/8 \rceil$ is the length required to select the leaf index idx_{leaf} within that XMSS tree (following the same calculation as SPHINCS⁺). The fourth argument is an arbitrary-length byte string, typically M or $M \parallel ctr$. For signing, the first L_{TFORS} bytes of the output are used as md (the TFORS index), the next $\lceil (h - h')/8 \rceil$ bytes are used for idx_{tree} , and the remaining $\lceil h'/8 \rceil$ bytes are used for idx_{leaf} .
- $\mathbf{PRF}(K_1, K_2, \mathbf{addr}) : \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^{32} \rightarrow \mathbb{B}^n$ derives deterministic pseudorandom values from two n -byte inputs and an address. In our scheme, it is used to derive atomic secret values from seeds and intermediate chain values, with $\mathbf{addr}$ separating the corresponding domains. This ensures the inputs to the PRF oracle are always distinct, thereby yielding outputs that are computationally independent.
- $\mathbf{T}_\ell(\mathbf{PK.seed}, \mathbf{addr}, M_\ell) : \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^{\ell n} \rightarrow \mathbb{B}^n$ compresses an ℓn -byte input into an n -byte output. It is used to compress the concatenation of all hash chain values into a single XMSS leaf.
- $\mathbf{H}(\mathbf{PK.seed}, \mathbf{addr}, M_2) : \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^{2n} \rightarrow \mathbb{B}^n$ is a special case of $\mathbf{T}_\ell$ that takes a $2n$ -byte message as input ($\ell = 2$). It is primarily used to compute interior nodes in the Merkle trees within TFORS.

- $\mathbf{F}(\mathbf{PK.seed}, \mathbf{addr}, M_1) : \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^n \rightarrow \mathbb{B}^n$ is a tweakable hash function that takes an n -byte message as input and produces an n -byte output. It is used in the hash chains and for leaf generation in TFORS.
- $\mathbf{H}_2(\mathbf{PK.seed}, M) : \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^* \rightarrow \mathcal{I}_{k,k'}$ maps the message to the TFORS index tuple, where

$$\mathcal{I}_{k,k'} = \{(i_1, \dots, i_k) \in \{0, \dots, k' - 1\}^k : i_a \neq i_b \text{ for } a \neq b\}.$$

These indices select the k TFORS secret-value sets used during signing. The values k, k' are fixed by the parameter set and are implicitly available to $\mathbf{H}_2$.

The concrete instantiations of these functions depend on the parameter set and are specified together with the full Phoenix scheme in Section 6.

2.4 Address and Member Functions

Address. Some hash-based functions in Phoenix take a 32-byte address **addr** as an additional input. The address encodes public domain-separation information, including the layer, tree position, function type, and local position of the value being computed. Different address values are used for different calls, so that calls to PRF, $\mathbf{T}_\ell$, $\mathbf{H}$, and $\mathbf{F}$ are separated across layers, trees, chains, and nodes.

Phoenix adopts an address format derived from the SPHINCS⁺ address structure [7], with extensions for the GWOTS+C and TFORS components. Each address consists of eight 4-byte words encoded as unsigned integers in big-endian byte order. The detailed layout of the address fields, including the layer address, tree address, type field, and component-specific fields, is given in Appendix A.

Member Functions. The algorithms use member-style notation to access and update structured byte arrays such as the 32-byte address **addr**. If a structure s contains a field X , then $s.\text{getX}()$ returns the value of that field, and $s.\text{setX}(Y)$ sets it to Y . If s contains multiple instances of a field X , then $s.\text{getX}(i)$ denotes the value of the i -th instance.

Address types determine the interpretation of the final 12 bytes of **addr**. Therefore, whenever the type field is changed, these bytes are cleared. We write $\mathbf{addr.setTypeAndClear}(Y)$ for the operation that sets the type field to Y and resets the final 12 bytes to zero. Symbolic constants such as GWOTS_CHAIN, GWOTSPK, HASHTREE, TFORSTREE, GWOTSPRF, TFORSPRF, and COMPRESS.GWOTS are used in the pseudocode to denote the corresponding integer type values.

The byte-level definitions of the address member functions are given in Appendix A.

3 Building Blocks 1: GWOTS+C

One-time signature is a basic building block of hash-based signature schemes. The earliest representative is the Lamport one-time signature [11], which signs

a message by revealing one secret value for each message bit. Although conceptually simple, Lamport signatures have large key and signature sizes. The Winternitz one-time signatures (WOTS/ WOTS+) reduce this overhead by encoding the message in base w and signing each digit with a hash chain [5] [9]. A checksum is appended so that an adversary cannot increase revealed chain values to obtain a valid signature on another message. Based on these schemes, several variants have been proposed. Constant-sum WOTS+ encodes message digits so that their sum is fixed, thereby reducing the need for an explicit checksum [16]. WOTS+C compresses SPHINCS⁺ signatures by omitting selected chains and using a checksum-based condition to preserve security [10]. More recently, variable-chain-length WOTS⁺ explores assigning different chain lengths within the same signature to obtain finer-grained tradeoffs between signature size and computation [14].

Our GWOTS+C construction follows this line of work. It keeps the hash-chain structure of WOTS+, but generalizes the checksum treatment by allowing a checksum-offset window rather than enforcing a single fixed checksum value. This gives a tunable tradeoff between signature size and signing cost, which is the main role of General WOTS+C in our scheme.

3.1 The Design Rationale of GWOTS+C

This section presents GWOTS+C, an optimized one-time signature scheme designed to provide a tunable tradeoff between signature size and signing time. The design is motivated by the observation that message-side compression and checksum-side constraints should be optimized jointly, since both affect the acceptance probability of nonce filtering and the final signature size.

At one extreme, WOTS+ uses no nonce filtering and signs the full checksum. At the other extreme, fixed-point WOTS+C uses rejection sampling to force the encoded message and checksum into a highly constrained form, allowing checksum-related chains, to be omitted. This yields shorter signatures, but the signer may need many trials because the digest must satisfy an exact target condition. GWOTS+C relaxes this exact-target constraint. The signer still uses nonce filtering to make selected message digits take compressible values, but the checksum is no longer forced to equal one predetermined value. Instead, the checksum is accepted whenever it falls within an admissible interval $[L, U]$, and the signature authenticates the exact offset within this interval. Figure 1 illustrates the structural difference between fixed-point WOTS+C and our generalization. In addition, GWOTS+C supports a mixed-width message profile, allowing different message blocks to use different Winternitz parameters, following the variable-chain-length idea studied in [14]. Together with the checksum-offset mechanism, this expands the design space while preserving the main compression benefit of chain omission.

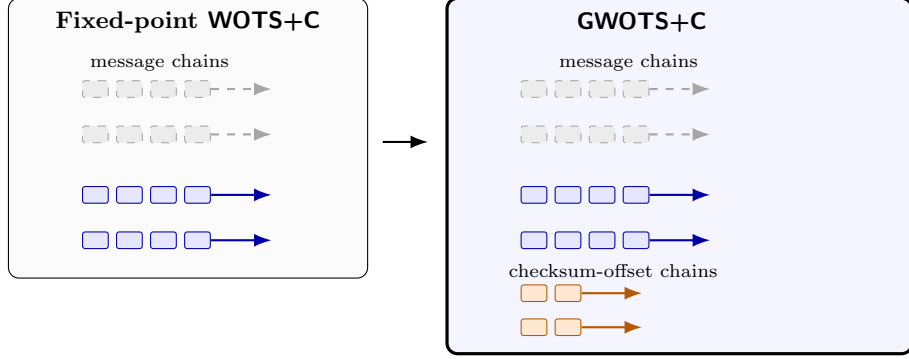


Figure 1: Hash-chain view of the idea. Fixed-point WOTS+C eliminates the checksum via nonce search, while GWOTS+C retains a short checksum-offset part with a window.

A Unified Cost Model for WOTS-Type structure. Our design of GWOTS+C is guided by a unified theory framework that captures the trade-off between signature size and signing cost. For a one-time signature module, signature size and running cost are tied together by a time-space tradeoff. We therefore track two basic quantities: NumOfChain which is the number of hash chains carried in the signature, and NumOfHashCall which is the total number of hash-function calls used by the module. Since each transmitted chain contributes one n -byte element, the signature size is linear in NumOfChain .

We further distinguish $\text{NumOfHashCall}_{\text{sign}}$ and $\text{NumOfHashCall}_{\text{verify}}$. Then we have

$$\text{NumOfHashCall}_{\text{sign}} \approx \sum_{i=1}^m \frac{2^{s_i} - 1}{2} + \sum_{j=1}^{d_{\text{off}}} \frac{2^{t_j} - 1}{2} + \frac{1}{p_{k_{\text{off}}}}$$

and

$$\text{NumOfHashCall}_{\text{verify}} \approx \sum_{i=1}^m \frac{2^{s_i} - 1}{2} + \sum_{j=1}^{d_{\text{off}}} \frac{2^{t_j} - 1}{2} + 1.$$

And the main factors are as follows:

- m chains are used to encode the hash of message bits,

$$\mathcal{S} = (s_1, \dots, s_m), \quad s_i \in \mathbb{Z}_{\geq 1}, \quad \sum_{i=1}^m s_i = 8n,$$

- the term $1/p_{k_{\text{off}}}$ accounts for the expected number of nonce-search iterations, where the average chain depths are computed under the uniform distribution on accepted digests. If the signer accepts exactly those digests for which

$L \leq C(a) \leq U$, then the acceptance probability of the checksum window is

$$p_{k_{\text{off}}} := \Pr[L \leq C(a) \leq U], \quad \mathbb{E}[\text{trials}] = \frac{1}{p_{k_{\text{off}}}}.$$

where trials denotes the number of nonce-search attempts required before the checksum falls inside the admissible window, the checksum $C(a) = \sum_{i=1}^m (2^{s_i} - 1 - a_i)$ for the induced digit vector $a = (a_1, \dots, a_m)$.

- d_{off} chains are used to encode the exact offset of the checksum from the lower bound of the acceptance window, and their widths are given by $\mathcal{T} = (t_1, \dots, t_{d_{\text{off}}})$.

Remark 3.1. The WOTS type schemes in this cost model can be viewed as following.

- WOTS+ is recovered by taking all $s_i = \log_2 w$, keeping the full checksum encoding, and setting $\mathbb{E}[\text{trials}] = 1$.
- Fixed-point WOTS+C is recovered by taking the same message profile but setting $d_{\text{off}} = 0$ (i.e., no checksum-offset chains are transmitted), forcing the checksum to a single fixed value via rejection sampling.
- The variable-chain-length WOTS+ is recovered by taking $d_{\text{off}} = 0$ and allowing the s_i to vary, but still forcing the checksum to a single fixed value.
- GWOTS+C keeps the same forward-only chain semantics as WOTS, but allows both the message profile and the checksum-offset profile to be mixed-width.

3.2 Parameter Selection Strategy

Distribution of the checksum. The checksum $C(a)$ is a sum of independent random variables $X_i := 2^{s_i} - 1 - a_i$, where each a_i is uniformly distributed in $\{0, 1, \dots, 2^{s_i} - 1\}$. The distribution of $C(a)$ is well approximated by a Gaussian law with mean μ and variance σ^2 where

$$\mu = \frac{1}{2} \sum_{i=1}^m (2^{s_i} - 1), \quad \sigma^2 = \frac{1}{12} \sum_{i=1}^m (2^{2s_i} - 1),$$

With the standard continuity correction, one obtains $\Pr[L \leq C(a) \leq U] \approx \Phi\left(\frac{U + \frac{1}{2} - \mu}{\sigma}\right) - \Phi\left(\frac{L - \frac{1}{2} - \mu}{\sigma}\right)$, where Φ denotes the cumulative distribution function of the standard normal distribution. In particular, if the acceptance window is chosen approximately symmetrically around the mean with half-width Δ , that is, $L \approx \mu - \Delta$ and $U \approx \mu + \Delta$, then $p_{k_{\text{off}}} \approx \Phi\left(\frac{\Delta + \frac{1}{2}}{\sigma}\right) - \Phi\left(\frac{-\Delta - \frac{1}{2}}{\sigma}\right) =$

$2\Phi\left(\frac{\Delta+\frac{1}{2}}{\sigma}\right) - 1$. Thus, for a fixed window width, centering the interval near μ captures the largest probability mass to first order.

Parameter search. The parameter search is organized around a simple optimization proxy. For a message profile $\mathcal{S} = (s_1, \dots, s_m)$, an offset profile $\mathcal{T} = (t_1, \dots, t_{d_{\text{off}}})$, and a centered checksum window of width $2^{k_{\text{off}}}$, for minimizing the expected number of hash function calls in signing, the objective function is

$$\text{Obj}(\mathcal{S}, \mathcal{T}, k_{\text{off}}) = \frac{1}{2} \sum_{i=1}^m (2^{s_i} - 1) + \frac{1}{2} \sum_{j=1}^{d_{\text{off}}} (2^{t_j} - 1) + \frac{1}{p_{k_{\text{off}}}},$$

subject to

$$\sum_{i=1}^m s_i = 8n, \quad \sum_{j=1}^{d_{\text{off}}} t_j = k_{\text{off}}, \quad m + d \leq \ell_{\text{max}}.$$

We run a search over all admissible message profiles $\mathcal{S}$, offset profiles $\mathcal{T}$, and window widths k_{off} to find the combination that minimizes the objective function under a preset max chain number ℓ_{max} . The resulting optimal parameters are then used to instantiate GWOTS+C for the desired security level.

3.3 The Algorithmic Specification

For the Key Generation of GWOTS+C, its specification is shown in Algorithm 1. For the Sign shown in Algorithm 2, a candidate counter ctr is used to compute the digest $H(ctr \| M)$, then parse it according to the mixed-width message profile $\mathcal{S}$. The signer iterates over ctr until the checksum satisfies $L \leq C(a) \leq U$. Once this condition holds, the message digits are signed using the standard WOTS chain computation. On the checksum side, instead of transmitting the full checksum encoding, the signer transmits only a short mixed-radix encoding of the offset $C(a) - L$.

For the verification which is shown in Algorithm 3, the verifier recomputes the digest from the transmitted counter and message, reconstructs the mixed-width digits, recomputes the checksum, checks that $L \leq C(a) \leq U$, and then verifies both the message chains and the checksum-offset chains against the public key.

Algorithm 1 GWOTS_KeyGen

Input: Private-key seed SK.seed , public-key seed PK.seed , base address addr , key-pair index idx

Output: One-time public key PK

```
1:  $\text{leafAddr} \leftarrow \text{addr}$ 
2: /* Set the public key address. */
3:  $\text{pkAddr} \leftarrow \text{addr}$ 
4:  $\text{pkAddr} \leftarrow \text{pkAddr.setTypeAndClear}(\text{GWOTSPK})$ 
5:  $\text{pkAddr} \leftarrow \text{pkAddr.setKeyPairAddress}(\text{idx})$ 
6: for  $i = 0$  to  $\ell - 1$  do
7:    $w_i \leftarrow \text{ChainWidth}(i)$ 
8:   /* Derive the secret value for the i-th GWOTS chain. */
9:    $\text{leafAddr} \leftarrow \text{leafAddr.setTypeAndClear}(\text{GWOTSPRF})$ 
10:   $\text{leafAddr} \leftarrow \text{leafAddr.setKeyPairAddress}(\text{idx})$ 
11:   $\text{leafAddr} \leftarrow \text{leafAddr.setChainAddress}(i)$ 
12:   $\text{leafAddr} \leftarrow \text{leafAddr.setHashAddress}(0)$ 
13:   $sk_i \leftarrow \text{PRF}(\text{PK.seed}, \text{SK.seed}, \text{leafAddr})$ 
14:  /* Switch to the hash-chain domain for the same key pair and chain. */
15:   $\text{leafAddr} \leftarrow \text{leafAddr.setTypeAndClear}(\text{GWOTS.CHAIN})$ 
16:   $\text{leafAddr} \leftarrow \text{leafAddr.setKeyPairAddress}(\text{idx})$ 
17:   $\text{leafAddr} \leftarrow \text{leafAddr.setChainAddress}(i)$ 
18:   $x \leftarrow sk_i$ 
19:  /* Advance the chain to its public endpoint. */
20:  for  $k = 0$  to  $w_i - 2$  do
21:     $\text{leafAddr} \leftarrow \text{leafAddr.setHashAddress}(k)$ 
22:     $x \leftarrow \text{F}(\text{PK.seed}, \text{leafAddr}, x)$ 
23:  end for
24:   $\text{tmp}[i] \leftarrow x$ 
25: end for
26: /* Compress all GWOTS chain endpoints into the one-time public key. */
27:  $\text{PK} \leftarrow \text{T}_\ell(\text{PK.seed}, \text{pkAddr}, \text{tmp})$ 
28: return PK
```

Algorithm 2 GWOTS.Sign

Input: Message M , private-key seed SK.seed , public-key seed PK.seed , base address addr , key-pair index idx , checksum lower bound L , checksum upper bound U

Output: Signature σ

```
1:  $ctr \leftarrow 0$ 
2: /* Search for a counter that makes the checksum admissible. */
3: repeat
4:    $ctr \leftarrow ctr + 1$ 
5:    $\mathbf{cAddr} \leftarrow \text{addr.setTypeAndClear}(\text{COMPRESS\_GWOTS})$ 
6:    $\mathbf{cAddr} \leftarrow \mathbf{cAddr.setKeyPairAddress}(\text{idx})$ 
7:    $\mathbf{cAddr} \leftarrow \mathbf{cAddr.setCounter}(ctr)$ 
8:    $md \leftarrow \mathbf{F}(\text{PK.seed}, \mathbf{cAddr}, M)$ 
9:    $a \leftarrow \text{Parse}_S(md)$ 
10:   $C \leftarrow \sum_{i=0}^{m-1} (2^{s_i} - 1 - a_i)$ 
11: until  $L \leq C \leq U$ 
12:  $\text{steps} \leftarrow a \parallel \text{Encode}_T(C - L)$ 
13: /* Generate the chain value at the selected step for each GWOTS chain. */
14: for  $i = 0$  to  $\ell - 1$  do
15:    $\mathbf{skAddr} \leftarrow \text{addr.setTypeAndClear}(\text{GWOTSPRF})$ 
16:    $\mathbf{skAddr} \leftarrow \mathbf{skAddr.setKeyPairAddress}(\text{idx})$ 
17:    $\mathbf{skAddr} \leftarrow \mathbf{skAddr.setChainAddress}(i)$ 
18:    $\mathbf{skAddr} \leftarrow \mathbf{skAddr.setHashAddress}(0)$ 
19:    $x \leftarrow \text{PRF}(\text{PK.seed}, \text{SK.seed}, \mathbf{skAddr})$ 
20:    $\mathbf{chainAddr} \leftarrow \text{addr.setTypeAndClear}(\text{GWOTS\_CHAIN})$ 
21:    $\mathbf{chainAddr} \leftarrow \mathbf{chainAddr.setKeyPairAddress}(\text{idx})$ 
22:    $\mathbf{chainAddr} \leftarrow \mathbf{chainAddr.setChainAddress}(i)$ 
23:   for  $k = 0$  to  $\text{steps}[i] - 1$  do
24:      $\mathbf{chainAddr} \leftarrow \mathbf{chainAddr.setHashAddress}(k)$ 
25:      $x \leftarrow \mathbf{F}(\text{PK.seed}, \mathbf{chainAddr}, x)$ 
26:   end for
27:    $\text{sig}[i] \leftarrow x$ 
28: end for
29: return  $\sigma \leftarrow ctr \parallel \text{sig}$   $\{ctr \text{ is fixed-length (4 bytes)}\}$ 
```

Algorithm 3 GWOTS_Verify

Input: Message M , signature σ , public key PK , public-key seed PK.seed , base address addr , key-pair index idx , checksum lower bound L , checksum upper bound U

Output: Boolean

```
1:  $\text{ctr} \leftarrow \sigma.\text{getCtr}()$ 
2:  $\text{sig} \leftarrow \sigma.\text{getChainValues}()$ 
3: /* Reconstruct the same message encoding used during signing. */
4:  $\text{cAddr} \leftarrow \text{addr}.\text{setTypeAndClear}(\text{COMPRESS\_GWOTS})$ 
5:  $\text{cAddr} \leftarrow \text{cAddr}.\text{setKeyPairAddress}(\text{idx})$ 
6:  $\text{cAddr} \leftarrow \text{cAddr}.\text{setCounter}(\text{ctr})$ 
7:  $\text{md} \leftarrow \mathbf{F}(\text{PK.seed}, \text{cAddr}, M)$ 
8:  $a \leftarrow \text{Parse}_S(\text{md})$ 
9:  $C \leftarrow \sum_{i=0}^{m-1} (2^{s_i} - 1 - a_i)$ 
10: if  $C < L$  or  $C > U$  then
11:   return false
12: end if
13:  $\text{steps} \leftarrow a \parallel \text{Encode}_{\mathcal{T}}(C - L)$ 
14: /* Advance each signature chain value to its public endpoint. */
15: for  $i = 0$  to  $\ell - 1$  do
16:    $w_i \leftarrow \text{ChainWidth}(i)$ 
17:    $x \leftarrow \text{sig}[i]$ 
18:    $\text{chainAddr} \leftarrow \text{addr}.\text{setTypeAndClear}(\text{GWOTS\_CHAIN})$ 
19:    $\text{chainAddr} \leftarrow \text{chainAddr}.\text{setKeyPairAddress}(\text{idx})$ 
20:    $\text{chainAddr} \leftarrow \text{chainAddr}.\text{setChainAddress}(i)$ 
21:   for  $k = \text{steps}[i]$  to  $w_i - 2$  do
22:      $\text{chainAddr} \leftarrow \text{chainAddr}.\text{setHashAddress}(k)$ 
23:      $x \leftarrow \mathbf{F}(\text{PK.seed}, \text{chainAddr}, x)$ 
24:   end for
25:    $\text{tmp}[i] \leftarrow x$ 
26: end for
27: /* Compress the recovered endpoints and compare with the public key. */
28:  $\text{pkAddr} \leftarrow \text{addr}.\text{setTypeAndClear}(\text{GWOTSPK})$ 
29:  $\text{pkAddr} \leftarrow \text{pkAddr}.\text{setKeyPairAddress}(\text{idx})$ 
30:  $\text{PK}' \leftarrow \mathbf{T}_{\ell}(\text{PK.seed}, \text{pkAddr}, \text{tmp})$ 
31: return  $\text{PK}' = \text{PK}$ 
```

Remark on Algorithms 1–3. The function $\text{ChainWidth}(i)$ returns the width w_i of the i -th GWOTS chain according to the message profile $\mathcal{S}$ and the checksum-offset profile $\mathcal{T}$. The parser Parse_S maps the digest to mixed-width message digits $a_i \in \{0, \dots, 2^{s_i} - 1\}$, while $\text{Encode}_{\mathcal{T}}(C - L)$ encodes the accepted checksum offset in the mixed radix determined by $\mathcal{T}$. The signature accessors $\sigma.\text{getCtr}()$ and $\sigma.\text{getChainValues}()$ return the counter and the chain values from $\sigma = \text{ctr} \parallel \text{sig}$, respectively.

3.4 Correctness and Security Analysis

Here we give a brief sketch of the security argument. The mixed-width construction preserves the monotonicity argument underlying standard WOTS-style security analyses [5, 9]. In particular, the set of accepted encodings forms an antichain under the coordinatewise order, which prevents an adversary from deriving one valid signature from another by only moving forward along public hash chains.

Proposition 3.1 (Antichain property of legal encodings). *Let $a = (a_1, \dots, a_m)$ be a mixed-width message-digit vector, and let*

$$C(a) = \sum_{i=1}^m (2^{s_i} - 1 - a_i)$$

be its checksum. For every accepted vector satisfying $L \leq C(a) \leq U$, define $\text{Enc}(a)$ as the concatenation of the message digits a and the canonical fixed-length mixed-radix encoding of $C(a) - L$ in bases $(2^{t_1}, \dots, 2^{t_{d_{\text{off}}}})$. If two accepted vectors a and a' satisfy

$$\text{Enc}(a) \leq \text{Enc}(a')$$

coordinatewise, then $\text{Enc}(a) = \text{Enc}(a')$.

Proof. From the message part we obtain $a_i \leq a'_i$ for every i , hence

$$C(a') - C(a) = - \sum_{i=1}^m (a'_i - a_i) \leq 0.$$

Thus $C(a') \leq C(a)$. On the other hand, the offset digits of $C(a) - L$ are coordinatewise no larger than those of $C(a') - L$, and every mixed-radix weight is positive, so the represented integer cannot decrease. Hence $C(a) - L \leq C(a') - L$, that is, $C(a) \leq C(a')$. Therefore $C(a) = C(a')$.

Substituting this equality back into the previous display gives

$$\sum_{i=1}^m (a'_i - a_i) = 0.$$

Since every summand is nonnegative, each summand must be zero, so $a'_i = a_i$ for all i . The offset digits then also coincide, and therefore $\text{Enc}(a) = \text{Enc}(a')$.

The one-time security of GWOTS+C. Based on Proposition 3.1, the GWOTS+C construction yields the same proof skeleton as for standard WOTS variants. A successful one-time forgery must either (i) produce the same legal encoding for a different digest, which reduces to target-collision resistance of the digest mapping, or (ii) move to a strictly smaller coordinate on at least one chain, which reduces to one-wayness of the corresponding hash chain [5]. The mixed-width generalization changes the coordinate ranges but not the proof pattern.

4 Building Blocks 2: Transposed FORS(TFORS)

Few-time signatures (FTS) are a basic component of stateless hash-based signature schemes. HORS [15] is the classical starting point: a message digest is parsed into k indices, and the signature reveals the corresponding k secret values from a larger collection of t values. This gives a simple few-time signing mechanism, but the public key contains t hash values and is therefore too large for direct use in a compact signature scheme. HORST [3], used in the original SPHINCS construction, addresses this by committing to these values with a Merkle tree, reducing the public key to a single root. The price is that the signature must include authentication information for the selected leaves. Since several selected leaves may lie in the same tree, their authentication paths can share internal nodes, and duplicate nodes can be removed. To address the index-collision and path-redundancy issues, PORS [2] maps the digest to k distinct indices and applies Octopus pruning to retain only the non-redundant authentication nodes. This avoids repeated selected indices and can significantly reduce the authentication part. However, the number of retained nodes depends on the selected leaf positions, so the signature length is naturally variable.

FORS, introduced in SPHINCS⁺ [4], partitions the secret values into k independent Merkle trees, each with $t = 2^a$ leaves. Signing reveals one secret value from each tree and includes its full authentication path. This avoids repeated indices by construction and gives a simple fixed-length signature, but it also prevents cross-tree path sharing. Consequently, the authentication cost grows linearly with k .

Recent work has explored how to recover pruning benefits while preserving fixed-size signatures. FORS+C [10] uses counter-based filtering to trade signing time for signature size, and also introduces an interleaved-tree arrangement for FORS leaves so that authentication paths can overlap and be pruned. However, the practical evaluation in FORS+C indicates that this interleaving optimization provides only limited additional savings for the considered parameter sets, and pruning alone naturally leads to variable-length signatures. PORS+FP [1] addresses this latter issue in a PORS-style scheme by repeatedly sampling the selected leaf indices until the Octopus-pruned authentication set is below a pre-defined bound.

4.1 Design Rationale

TFORS follows this line of work, but uses the transposed layout as part of a standalone few-time signature module and combines it explicitly with forced pruning. The goal is to repeatedly sample the selected leaf indices until the Octopus-pruned set can be used. What's different from PORS+FP is that the messages to be signed are organized through transposed FORS, meaning that similar messages

will collide earlier. This allows us to estimate the length of TFORS solely based on the messages to be signed, avoiding complex calculations.

Instead of following the standard FORS rule of selecting one leaf from each of k fixed trees, TFORS defines k' secret-value sets and, for each signature, selects k distinct sets with $k < k'$. This k -out-of- k' selection rule gives the construction an additional design parameter: increasing k' reduces set-level reuse across signatures, while still revealing only k secret values per signature. This lowers the probability that previously observed signatures cover all secret values required for a new forgery.

TFORS then transposes the placement of secret values. While FORS assigns each set to an independent tree, TFORS arranges all $T = k' \cdot t$ secret values as leaves of a *single* complete Merkle tree of height

$$h_{\text{TFORS}} = a + \log_2 k' .$$

Leaves are indexed so that secret values sharing the same intra-set position i are placed in contiguous blocks, each block naturally forming a complete subtree of height $\log_2 k'$. This transposed layout makes authentication paths of selected leaves overlap more often, so the Octopus algorithm can remove more redundant nodes. Because k' is a power of two, the overall tree is perfectly balanced, which avoids padding, keeps tree indexing simple, and makes the Octopus and Inverse Octopus procedures easier to specify.

Finally, TFORS incorporates forced pruning into the module specification: the signer varies a counter until the pruned authentication set satisfies a fixed size bound. At the expense of additional signing trials, this mechanism prunes the authentication path to a length below a preset maximum. Consequently, TFORS facilitates a configurable trade-off among signing time, signature size, and the probability of set reuse.

4.2 The Octopus Algorithm and Inverse Octopus Algorithm

The Octopus algorithm, proposed in PORS [2], is a Merkle-path pruning procedure for a set of selected leaves. TFORS uses the same pruning principle, but adapts it to the transposed-tree structure described above. The main refinement is that pruning is split into three deterministic subroutines. First, `Octopus Auth Indices` computes only the required authentication positions as $(\text{level}, \text{index})$ pairs. This index-only step can be used during forced-pruning search to test whether a candidate digest satisfies the prescribed authentication-size bound before materializing the authentication nodes. Second, `Octopus` uses these positions to extract the corresponding node values from a complete binary transposed tree. Third, *Inverse Octopus* gives the verifier-side reconstruction procedure using the same ordered authentication representation. Because

TFORS chooses the transposed tree size to be a power of two, these routines do not require padding cases, and the sibling arithmetic remains uniform.

Concretely, the three routines are as follows:

Algorithm 4 (Octopus Auth Indices). Given a sorted list of distinct leaf indices idx and tree height h_{TFORS} , this routine computes the ordered list $\mathcal{I}$ of missing sibling positions required for authentication. It processes the tree level by level: if two active siblings are both present, they move up to their parent without adding an authentication node; otherwise, the missing sibling position is appended to $\mathcal{I}$.

Algorithm 5 (Octopus). This routine materializes the compressed authentication hashes. It first calls Algorithm 4 to obtain $\mathcal{I}$, then traverses the complete TFORS subtree. Whenever a computed node matches an entry of $\mathcal{I}$, its hash is stored in the corresponding position of **AUTH**. The routine returns the ordered authentication hash list **AUTH** and the Merkle root.

Algorithm 6 (Inverse Octopus). This routine reconstructs the Merkle root during verification. Starting from the selected leaf hashes in $\mathcal{C}$, it processes each level upward. When two siblings are both present, they are hashed together directly; when one sibling is missing, its hash is read from **AUTH** in the same order generated by Algorithm 4. The process continues until the unique root node is obtained.

Both algorithms are invoked directly in **TFORS.sign** (Algorithm 10) and **TFORS.pkFromSig** (Algorithm 11) to prune duplicate nodes during signature generation and recover the Merkle root during verification.

With this pruning machinery in hand, we now present the complete TFORS construction in the next subsection.

4.3 The TFORS Scheme

In this section, we describe Transposed FORS (TFORS), a few-time signature scheme. TFORS transposes the placement of secret values used in FORS-style constructions. Instead of assigning each selected set to an independent Merkle tree, TFORS groups secret values with the same intra-set position across different sets into a common transposed subtree. This layout creates overlap among authentication paths and allows the Octopus algorithm to remove redundant authentication nodes. Forced pruning is then used to ensure that the pruned authentication part satisfies a fixed size bound. The full TFORS construction is given in Algorithms 7 through 11.

TFORS Key Pair. TFORS is parameterized by three integers k' , k , and $t = 2^a$, where k' denotes the number of secret-value sets, each containing t secret values. The parameter k specifies the number of sets selected during signing (with $k < k'$; this ensures that not all sets are used in a single signature). The total number

Algorithm 4 Octopus.Auth.Indices

Input: Sorted distinct leaf indices idx , tree height h_{TFORS}

Output: Ordered authentication index list $\mathcal{I}$ of $(level, index)$ pairs

```
1:  $curr \leftarrow idx, \mathcal{I} \leftarrow \emptyset$ 
2: for  $level = 0$  to  $h_{\text{TFORS}} - 1$  do
3:    $par \leftarrow \emptyset, i \leftarrow 0$ 
4:   while  $i < |curr|$  do
5:      $node\_idx \leftarrow curr[i]$ 
6:      $sib\_idx \leftarrow node\_idx \oplus 1$ 
7:      $par \leftarrow par \parallel \lfloor node\_idx/2 \rfloor$ 
8:     if  $i + 1 < |curr|$  and  $curr[i + 1] = sib\_idx$  then
9:        $i \leftarrow i + 2$ 
10:    else
11:       $\mathcal{I} \leftarrow \mathcal{I} \parallel [(level, sib\_idx)]$ 
12:       $i \leftarrow i + 1$ 
13:    end if
14:  end while
15:   $curr \leftarrow \text{Deduplicate}(par)$ 
16: end for
17: return  $\mathcal{I}$ 
```

of secret values is therefore $T = k' \cdot t$. Specifically, TFORS comprises $T = k' \cdot t$ secret values of length n bytes, denoted $\mathbf{sk} = \{s_{i,j}\}_{i \in [t], j \in [k']}$. These values are pseudorandomly generated from $\mathbf{SK}.\text{seed}$ and an address. All T secret values are placed as leaves of a single complete Merkle tree of height $h_{\text{TFORS}} = a + \log_2 k'$. The leaf index of a secret value with intra-set position $i \in [0, t - 1]$ and set index $j \in [0, k' - 1]$ is defined as

$$\text{leaf_idx} = i \cdot k' + j.$$

This ordering guarantees that secret values sharing the same i form a contiguous subtree of height $\log_2 k'$, which facilitates pruning. The root of this tree is the TFORS public key and can be obtained by calling `TFORS_node` with target index 0 and height h_{TFORS} .

TFORS Signature Generation. To ensure that the pruned signature size remains below a predefined bound, TFORS employs a forced pruning mechanism. This mechanism is executed within the `Phoenix.sign` algorithm (Algorithm 18). Once a suitable message digest md is obtained, TFORS signs an L -byte digest where $L = \max(n, \lceil k \cdot a/8 \rceil)$. The first n bytes of md are processed by a function $\mathbf{H}_2$ (e.g., a cryptographic hash function) that outputs k distinct integers in the range $[0, k' - 1]$, thereby selecting k distinct secret-key sets from the k' available sets. The digest also provides $k \cdot a$ bits, parsed as k chunks of a bits each. Each

Algorithm 5 Octopus

Input: Sorted distinct leaf indices idx , tree height h_{TFORS} , address **addr**, secret seed SK.seed, public seed PK.seed

Output: Ordered authentication hash list AUTH, Merkle root **root**

```
1:  $\mathcal{I} \leftarrow \text{Octopus\_Auth\_Indices}(idx, h_{\text{TFORS}})$ 
2:  $\mathcal{S}[0, \dots, h_{\text{TFORS}} - 1] \leftarrow \mathbf{0}^n$ 
3:  $max\_idx \leftarrow (1 \ll h_{\text{TFORS}}) - 1$ 
4: AUTH  $\leftarrow \emptyset$ 
5: for  $r = 0$  to  $max\_idx$  do
6:   addr.setTreeHeight(0)
7:   addr.setTreeIndex( $r$ )
8:   addr.setType(TFORS_PRF)
9:    $sk \leftarrow \text{PRF}(\text{PK.seed}, \text{SK.seed}, \mathbf{addr})$ 
10:  addr.setType(TFORS_TREE)
11:   $curr \leftarrow \mathbf{F}(\text{PK.seed}, \mathbf{addr}, sk)$ 
12:   $node\_idx \leftarrow r$ 
13:  for  $\ell = 0$  to  $h_{\text{TFORS}}$  do
14:    if  $\ell = h_{\text{TFORS}}$  then
15:      root  $\leftarrow curr$ 
16:      return AUTH, root
17:    end if
18:    for  $j = 0$  to  $|\mathcal{I}| - 1$  do
19:      if  $\mathcal{I}[j].\text{level} = \ell$  and  $\mathcal{I}[j].\text{index} = node\_idx$  then
20:        AUTH[ $j$ ]  $\leftarrow curr$ 
21:        break
22:      end if
23:    end for
24:    if  $(node\_idx \text{ AND } 1) = 0$  and  $r < max\_idx$  then
25:      break
26:    end if
27:    addr.setTreeHeight( $\ell + 1$ )
28:    addr.setTreeIndex( $\lfloor node\_idx / 2 \rfloor$ )
29:     $curr \leftarrow \mathbf{H}(\text{PK.seed}, \mathbf{addr}, \mathcal{S}[\ell] \parallel curr)$ 
30:     $node\_idx \leftarrow \lfloor node\_idx / 2 \rfloor$ 
31:  end for
32:   $\mathcal{S}[\ell] \leftarrow curr$ 
33: end for
```

chunk is interpreted as an integer in $[0, t - 1]$ and selects the intra-set position for the corresponding selected set. The helper `SortIndices( $\cdot$ )` sorts the resulting leaf indices in ascending order. Since $\mathbf{H}_2$ outputs distinct set indices, the final leaf indices are also distinct.

Algorithm 6 Inverse Octopus

Input: Sorted distinct leaf indices idx , tree height h_{TFORS} , ordered authentication hashes AUTH , leaf hashes $leaf$, address $\mathbf{addr}$, public seed PK.seed

Output: Merkle root $\mathbf{root}$

```
1:  $\mathcal{C} \leftarrow \emptyset$ 
2: for  $i = 0$  to  $|idx| - 1$  do
3:    $\mathcal{C} \leftarrow \mathcal{C} \parallel [(idx[i], leaf[i])]$ 
4: end for
5:  $auth\_ptr \leftarrow 0$ 
6: for  $level = 0$  to  $h_{\text{TFORS}} - 1$  do
7:    $\mathcal{P} \leftarrow \emptyset, i \leftarrow 0$ 
8:   while  $i < |\mathcal{C}|$  do
9:      $j \leftarrow \mathcal{C}[i].\text{index}$ 
10:     $par\_idx \leftarrow \lfloor j/2 \rfloor$ 
11:    if  $i + 1 < |\mathcal{C}|$  and  $\mathcal{C}[i + 1].\text{index} = (j \oplus 1)$  then
12:       $left \leftarrow \mathcal{C}[i].\text{hash}, right \leftarrow \mathcal{C}[i + 1].\text{hash}$ 
13:       $i \leftarrow i + 2$ 
14:    else if  $(j \text{ AND } 1) = 1$  then
15:       $left \leftarrow \text{AUTH}[auth\_ptr], right \leftarrow \mathcal{C}[i].\text{hash}$ 
16:       $auth\_ptr \leftarrow auth\_ptr + 1, i \leftarrow i + 1$ 
17:    else
18:       $left \leftarrow \mathcal{C}[i].\text{hash}, right \leftarrow \text{AUTH}[auth\_ptr]$ 
19:       $auth\_ptr \leftarrow auth\_ptr + 1, i \leftarrow i + 1$ 
20:    end if
21:     $\mathbf{addr.setTreeHeight}(level + 1)$ 
22:     $\mathbf{addr.setTreeIndex}(par\_idx)$ 
23:     $\mathcal{P} \leftarrow \mathcal{P} \parallel [(par\_idx, \mathbf{H}(\text{PK.seed}, \mathbf{addr}, left \parallel right))]$ 
24:  end while
25:   $\mathcal{C} \leftarrow \mathcal{P}$ 
26: end for
27:  $\mathbf{root} \leftarrow \mathcal{C}[0].\text{hash}$ 
28: return  $\mathbf{root}$ 
```

Algorithm 7 TFORS_skGen

Input: Secret seed SK.seed , public seed PK.seed , address $\mathbf{addr}$, leaf index idx

Output: TFORS Private Key $\mathbf{sk}$

```
1:  $\mathbf{skAddr} \leftarrow \mathbf{addr}$ 
2:  $\mathbf{skAddr.setTreeHeight}(0)$ 
3:  $\mathbf{skAddr.setTreeIndex}(idx)$ 
4:  $\mathbf{skAddr.setType}(\text{TFORS\_PRF})$ 
5:  $\mathbf{sk} \leftarrow \text{PRF}(\text{PK.seed}, \text{SK.seed}, \mathbf{skAddr})$ 
6: return  $\mathbf{sk}$ 
```

Algorithm 8 TFORS_node

Input: Secret seed SK.seed, public seed PK.seed, address **addr**, target node index i , target node height z

Output: *node*

```
1: if  $z = 0$  then
2:    $sk \leftarrow \text{TFORS\_skGen}(\text{SK.seed}, \text{PK.seed}, \mathbf{addr}, i)$ 
3:    $\mathbf{addr.setTreeHeight}(0)$ 
4:    $\mathbf{addr.setTreeIndex}(i)$ 
5:    $\mathbf{addr.setType}(\text{TFORS\_TREE})$ 
6:    $node \leftarrow \mathbf{F}(\text{PK.seed}, \mathbf{addr}, sk)$ 
7: else
8:    $lnode \leftarrow \text{TFORS\_node}(\text{SK.seed}, \text{PK.seed}, \mathbf{addr}, 2i, z - 1)$ 
9:    $rnode \leftarrow \text{TFORS\_node}(\text{SK.seed}, \text{PK.seed}, \mathbf{addr}, 2i + 1, z - 1)$ 
10:   $\mathbf{addr.setTreeHeight}(z)$ 
11:   $\mathbf{addr.setTreeIndex}(i)$ 
12:   $node \leftarrow \mathbf{H}(\text{PK.seed}, \mathbf{addr}, lnode \parallel rnode)$ 
13: end if
14: return node
```

Algorithm 9 message_to_indices

Input: Message digest md , public seed PK.seed

Output: indices *indices*

```
1:  $indices1 \leftarrow H_2(md, \text{PK.seed})$ 
2:  $offset \leftarrow 0$ 
3: for  $i = 0$  to  $k - 1$  do
4:    $indices2[i] \leftarrow 0$ 
5:   for  $j = 0$  to  $a - 1$  do
6:      $indices2[i] \leftarrow indices2[i] \text{ OR } ((md[offset \gg 3] \gg (offset \text{ AND } 7)) \text{ AND } 1) \ll j$ 
7:      $offset \leftarrow offset + 1$ 
8:   end for
9:    $indices[i] \leftarrow indices1[i] + k' \times indices2[i]$ 
10: end for
11:  $\text{SortIndices}(indices)$ 
12: return indices
```

During verification, the same message digest is processed identically to recompute the indices, and the root node reconstructed from the signature is compared against the TFORS public key. Notably, the a -bit chunks (and thus the selected intra-set indices) may repeat across different sets. Such repetitions increase the overlap among authentication paths, enabling more effective pruning via the Octopus algorithm. The signature consists of the selected secret values together with their pruned authentication paths, which are computed using the Octopus algorithm. The verifier reconstructs a root PK' of the aggregated Merkle tree

using the provided authentication paths. Verification succeeds if PK' equals the TFORS public key PK , and fails otherwise.

Algorithm 10 TFORS_sign

Input: Secret seed $SK.seed$, public seed $PK.seed$, address $\mathbf{addr}$, leaf indices $indices$.

Output: TFORS signature SIG_{TFORS} , TFORS root $\mathbf{root}$.

```

1:  $SIG_{TFORS} \leftarrow \text{NULL}$ 
2: for  $i$  from 0 to  $k - 1$  do
3:    $SIG_{TFORS} \leftarrow SIG_{TFORS} \parallel \text{TFORS\_skGen}(SK.seed, PK.seed, \mathbf{addr}, indices[i])$ 
4: end for
5:  $(\text{AUTH}, \mathbf{root}) \leftarrow \text{Octopus}(indices, h_{TFORS}, \mathbf{addr}, SK.seed, PK.seed)$ 
6:  $SIG_{TFORS} \leftarrow SIG_{TFORS} \parallel \text{AUTH}$ 
7: return  $SIG_{TFORS}, \mathbf{root}$ 

```

Algorithm 11 TFORS_pkFromSig

Input: Public seed $PK.seed$, Address $\mathbf{addr}$, Message digest md , TFORS signature SIG_{TFORS} .

Output: TFORS public key pk .

```

1:  $indices \leftarrow \text{message\_to\_indices}(md, PK.seed)$ 
2: for  $i$  from 0 to  $k - 1$  do
3:    $sk \leftarrow SIG_{TFORS}.getSK[i]$ 
4:    $\mathbf{addr}.setTreeHeight(0)$ 
5:    $\mathbf{addr}.setTreeIndex(indices[i])$ 
6:    $\mathbf{addr}.setType(TFORS\_TREE)$ 
7:    $node[i] \leftarrow F(PK.seed, \mathbf{addr}, sk)$ 
8: end for
9:  $\text{AUTH} \leftarrow SIG_{TFORS}.getAUTH()$ 
10: return  $\text{Inverse\_Octopus}(indices, h_{TFORS}, \text{AUTH}, node, \mathbf{addr}, PK.seed)$ 

```

The derivation of indices, along with the forced-pruning loop, is performed in `Phoenix_sign_internal` (Algorithm 18). Consequently, `TFORS_sign` receives the precomputed leaf indices $indices$ directly, avoiding redundant recomputation across iterations.

4.4 Security Analysis

The security of TFORS relies on the standard hash-function assumptions used for hash-based signatures, including pseudorandom secret-key generation, one-wayness of the leaf hash function, and collision resistance of the Merkle-tree

compression function. In addition, as in FORS-style few-time signatures, one must bound the probability that an adversary can assemble a new valid TFORS signature from secret values revealed in previous signatures. The analysis below focuses on the interleaved target subset resilience (ITSR) of the message digest function [4]. ITSR captures the difficulty for an adversary, after observing γ legitimate signatures, of finding a new message digest for which every one of the k selected leaf indices has appeared in at least one previous signature. Unless stated otherwise, the probabilities below refer to the unconditioned TFORS selection rule before applying the forced-pruning acceptance test.

Consider a secret-value set of size $t = 2^a$. Suppose this set has appeared in γ observed signatures a total of r times. Assuming that each such occurrence reveals an independently uniform intra-set position, the probability that a new message digest's chosen secret value in this set has already been covered by previous signatures is:

$$P_{\text{cover}}(r) = 1 - \left(1 - \frac{1}{t}\right)^r.$$

A new TFORS signature selects k distinct secret-value sets from the k' available sets without replacement; denote the selected sets as $S \subseteq \{1, \dots, k'\}$. For each selected set $j \in S$, let γ_j be the number of times this set has appeared in the previous γ signatures. Conditioned on $(\gamma_j)_{j \in S}$, the probability that all k required secret values have already been revealed is the product of the coverage probabilities for each selected set:

$$P_{\text{success}}((\gamma_j)_{j \in S}) = \prod_{j \in S} P_{\text{cover}}(\gamma_j) = \prod_{j \in S} \left[1 - \left(1 - \frac{1}{t}\right)^{\gamma_j}\right].$$

Let γ_i be the number of times secret-key set i appears in γ signatures, where each signature selects k distinct sets uniformly from k' . Marginally,

$$\gamma_i \sim \text{Binomial}\left(\gamma, \frac{k}{k'}\right), \quad i = 1, \dots, k'.$$

More precisely, each previous signature contributes an incidence vector of Hamming weight k , and γ_i is the sum of the i -th coordinates of these incidence vectors. Hence the joint vector $(\gamma_1, \dots, \gamma_{k'})$ is not multinomial: each trial selects k distinct sets rather than a single category. Its coordinates have the fixed total $\sum_{i=1}^{k'} \gamma_i = k\gamma$ and satisfy the standard negative-dependence property associated with sampling without replacement.

The set S is chosen randomly by the new message digest, independently of the previous signatures. Therefore, the overall ITSR coverage probability can be bounded by taking the expectation over both S and the selection-count vector:

$$P_{\text{TFORS}} = \mathbb{E}_{S, (\gamma_1, \dots, \gamma_{k'})} \left[\prod_{j \in S} \left(1 - \left(1 - \frac{1}{t} \right)^{\gamma_j} \right) \right].$$

Let

$$f(x) = 1 - \left(1 - \frac{1}{t} \right)^x.$$

Since f is nonnegative and increasing for $x \geq 0$, and the γ_j 's are negatively dependent, the product of expectations provides an upper bound on the expectation of the product. Thus, for any fixed S of size k ,

$$\mathbb{E} \left[\prod_{j \in S} f(\gamma_j) \right] \leq \prod_{j \in S} \mathbb{E}[f(\gamma_j)] = (\mathbb{E}[f(\gamma_1)])^k.$$

With $\gamma_1 \sim \text{Binomial}(\gamma, k/k')$, we compute:

$$\mathbb{E}[f(\gamma_1)] = 1 - \mathbb{E} \left[\left(1 - \frac{1}{t} \right)^{\gamma_1} \right] = 1 - \left(1 - \frac{k}{k'} \cdot \frac{1}{t} \right)^\gamma = 1 - \left(1 - \frac{k}{k't} \right)^\gamma.$$

Substituting this result gives an upper bound on the ITSR coverage probability for the unconditioned TFORS selection rule:

$$P_{\text{TFORS}} \leq \left[1 - \left(1 - \frac{k}{k't} \right)^\gamma \right]^k.$$

This bound is conservative: it overestimates the adversary's coverage probability (i.e., provides an upper bound on P_{TFORS}) and therefore yields a lower bound on the scheme's security. It should be read as the target-subset coverage bound used for parameter selection, not as a complete EU-CMA proof by itself. The full signature-scheme security also relies on the hash-function assumptions above and on the treatment of the forced-pruning acceptance predicate in the overall signing procedure.

5 Building Block 3: XMSS and Hypertree

Phoenix follows the classical XMSS and hypertree framework used in hash-based signatures [4, 6, 7, 13]. At the XMSS level, each tree authenticates a collection of one-time public keys by means of Merkle authentication paths. At the hypertree level, multiple XMSS trees are stacked so that the root of one layer is signed by an XMSS key pair in the next layer, and the public key contains only the root of the top XMSS tree. Phoenix keeps this standard authentication structure unchanged. The difference lies in the leaf-authentication modules: Phoenix

uses TFORS at the bottom layer and GWOTS+C inside each XMSS tree. Consequently, the Merkle authentication path computation and root reconstruction follow the standard XMSS/hypertree structure, while each XMSS signature carries a GWOTS+C signature and the 4-byte counter used by the GWOTS+C checksum-window search.

5.1 XMSS Layer

An XMSS tree in Phoenix authenticates GWOTS+C public keys. Algorithm 12 computes an XMSS subtree root, Algorithm 13 generates the XMSS signature used in one hypertree layer, and Algorithm 14 reconstructs the corresponding XMSS root from a signature.

Algorithm 12 XMSS_node

Input: Secret seed SK.seed, start index i , height z , public seed PK.seed, address **addr**

Output: Root node of the subtree

```

1: if  $z = 0$  then
2:   addr.setKeyPairAddress( $i$ )
3:    $pk \leftarrow \text{GWOTS\_KeyGen}(\text{SK.seed}, \text{PK.seed}, \mathbf{addr})$ 
4:   addr.setTypeAndClear(GWOTSPK)
5:   addr.setKeyPairAddress( $i$ )
6:    $node \leftarrow \mathbf{T}_\ell(\text{PK.seed}, \mathbf{addr}, pk)$ 
7:   return  $node$ 
8: else
9:    $left \leftarrow \text{XMSS\_node}(\text{SK.seed}, 2i, z - 1, \text{PK.seed}, \mathbf{addr})$ 
10:   $right \leftarrow \text{XMSS\_node}(\text{SK.seed}, 2i + 1, z - 1, \text{PK.seed}, \mathbf{addr})$ 
11:  addr.setTypeAndClear(HASHTREE)
12:  addr.setTreeHeight( $z$ )
13:  addr.setTreeIndex( $i$ )
14:   $node \leftarrow \mathbf{H}(\text{PK.seed}, \mathbf{addr}, left \parallel right)$ 
15:  return  $node$ 
16: end if

```

Algorithm 13 XMSS_sign

Input: Message M , secret seed $SK.seed$, leaf index idx , public seed $PK.seed$, address $addr$

Output: XMSS signature SIG_{XMSS} and XMSS root $root$

```
1:  $addr.setKeyPairAddress(idx)$ 
2:  $(\sigma, ctr_{GWOTS+C}) \leftarrow GWOTS\_sign(M, SK.seed, PK.seed, addr)$  {The values  $U, L$  are
   fixed by the parameter set, parse the result of  $GWOTS\_sign$  as  $(\sigma, ctr_{GWOTS+C})$ 
   where  $ctr_{GWOTS+C}$  occupies the first 4 bytes of the concatenation.}
3:  $AUTH \leftarrow \emptyset$ 
4: for  $j$  from 0 to  $h' - 1$  do
5:    $s \leftarrow \lfloor idx/2^j \rfloor \oplus 1$ 
6:    $node \leftarrow XMSS\_node(SK.seed, s, j, PK.seed, addr)$ 
7:    $AUTH.append(node)$ 
8: end for
9:  $root \leftarrow XMSS\_pkFromSig(idx, (\sigma, ctr_{GWOTS+C}, AUTH), M, PK.seed, addr)$ 
10:  $SIG_{XMSS} \leftarrow (\sigma, AUTH, ctr_{GWOTS+C})$ 
11: return  $(SIG_{XMSS}, root)$ 
```

Algorithm 14 XMSS_pkFromSig

Input: Leaf index idx , XMSS signature SIG_{XMSS} , message M , public seed $PK.seed$, address $addr$

Output: Root node

```
1: Parse  $SIG_{XMSS}$  as  $(\sigma, AUTH, ctr_{GWOTS+C})$ 
2:  $addr.setKeyPairAddress(idx)$ 
3:  $pk \leftarrow GWOTS\_pkFromSig(\sigma, M, ctr_{GWOTS+C}, PK.seed, addr)$ 
4:  $addr.setTypeAndClear(GWOTSPK)$ 
5:  $addr.setKeyPairAddress(idx)$ 
6:  $node \leftarrow T_\ell(PK.seed, addr, pk)$ 
7:  $addr.setTypeAndClear(HASHTREE)$ 
8: for  $j$  from 0 to  $h' - 1$  do
9:    $addr.setTreeHeight(j + 1)$ 
10:   $addr.setTreeIndex(\lfloor idx/2^{j+1} \rfloor)$ 
11:  if  $\lfloor idx/2^j \rfloor \bmod 2 = 0$  then
12:     $node \leftarrow H(PK.seed, addr, node \parallel AUTH[j])$ 
13:  else
14:     $node \leftarrow H(PK.seed, addr, AUTH[j] \parallel node)$ 
15:  end if
16: end for
17: return  $node$ 
```

5.2 Hypertree

The hypertree layer of Phoenix follows the standard hypertree construction [4, 7]. The only changes are in the authenticated components: Phoenix signs a TFORS public key at the bottom layer and uses GWOTS+C public keys as XMSS leaves. The resulting hypertree pseudocode is shown below for completeness. The address helper functions `copySubtreeAddress`, `setLayerAddress`, etc. follow the SPHINCS⁺ address model and are defined in Appendix A.

Algorithm 15 HT_sign

Input: Message to be signed msg , secret seed $SK.seed$, public seed $PK.seed$, tree index idx_{tree} , leaf index idx_{leaf}

Output: Hypertree signature SIG_{HT}

```

1:  $SIG_{HT} \leftarrow \emptyset$ 
2:  $root \leftarrow msg$ 
3:  $treeAddr \leftarrow \text{toByte}(0, 32)$ 
4:  $otsAddr \leftarrow \text{toByte}(0, 32)$ 
5: for  $j$  from 0 to  $d - 1$  do
6:    $treeAddr.setLayerAddress(j)$ 
7:    $treeAddr.setTreeAddress(idx_{tree})$ 
8:    $otsAddr.copySubtreeAddress(treeAddr)$ 
9:    $otsAddr.setKeyPairAddress(idx_{leaf})$ 
10:   $(SIG_{XMSS}, root) \leftarrow XMSS\_sign(root, SK.seed, idx_{leaf}, PK.seed, otsAddr)$ 
11:   $SIG_{HT}.append(SIG_{XMSS})$ 
12:   $idx_{leaf} \leftarrow idx_{tree} \bmod 2^{h'}$ 
13:   $idx_{tree} \leftarrow \lfloor idx_{tree} / 2^{h'} \rfloor$ 
14: end for
15: return  $SIG_{HT}$ 
```

Algorithm 16 HT.verify

Input: Message to be verified msg , hypertree signature SIG_{HT} , public seed $PK.seed$, tree index idx_{tree} , leaf index idx_{leaf} , public root $PK.root$

Output: **ACCEPT OR REJECT**

```
1:  $node \leftarrow msg$ 
2:  $treeAddr \leftarrow toByte(0, 32)$ 
3:  $otsAddr \leftarrow toByte(0, 32)$ 
4: for  $j$  from 0 to  $d - 1$  do
5:   Parse  $SIG_{HT}[j]$  as  $SIG_{XMSS}$ 
6:    $treeAddr.setLayerAddress(j)$ 
7:    $treeAddr.setTreeAddress(idx_{tree})$ 
8:    $otsAddr.copySubtreeAddress(treeAddr)$ 
9:    $otsAddr.setKeyPairAddress(idx_{leaf})$ 
10:   $node \leftarrow XMSS\_pkFromSig(idx_{leaf}, SIG_{XMSS}, node, PK.seed, otsAddr)$ 
11:   $idx_{leaf} \leftarrow idx_{tree} \bmod 2^{h'}$ 
12:   $idx_{tree} \leftarrow \lfloor idx_{tree} / 2^{h'} \rfloor$ 
13: end for
14: if  $node = PK.root$  then
15:   return ACCEPT
16: else
17:   return REJECT
18: end if
```

6 Phoenix Scheme

Phoenix is a stateless hash-based signature scheme built from three layers: a bottom-layer few-time signature module, an XMSS-based authentication layer, and a hypertree that combines multiple XMSS trees into a single public root. Following the general stateless hash-based framework, Phoenix uses message-derived indices to select signing instances and therefore does not require mutable signing state.

The main design difference lies in the two signature modules used inside this framework. At the bottom layer, Phoenix uses TFORS instead of the standard FORS component. Inside each XMSS tree, Phoenix uses GWOTS+C instead of WOTS+ [9]. The XMSS and hypertree authentication structure remains standard [6,13], while the TFORS and GWOTS+C components provide the signature-size optimizations described in the previous sections.

6.1 Key Generation

Phoenix key generation samples three independent n -byte values: the secret seed $SK.seed$, the message-randomization key $SK.prf$, and the public seed $PK.seed$.

The public seed is used in the hash-function calls to separate different public domains. The hypertree root PK.root is computed as the root of the top-layer XMSS tree. The resulting secret key is $(\text{SK.seed}, \text{SK.prf}, \text{PK.seed}, \text{PK.root})$, and the public key is $(\text{PK.seed}, \text{PK.root})$.

Algorithm 17 Phoenix_keygen

Input: Security parameter.

Output: Secret key SK, public key PK

- 1: $\text{SK.seed} \leftarrow_R \mathbb{B}^n$
 - 2: $\text{SK.prf} \leftarrow_R \mathbb{B}^n$
 - 3: $\text{PK.seed} \leftarrow_R \mathbb{B}^n$
 - 4: $\mathbf{addr} \leftarrow \text{toByte}(0, 32)$
 - 5: $\mathbf{addr.setLayerAddress}(d - 1)$
 - 6: $\text{PK.root} \leftarrow \text{XMSS_node}(\text{SK.seed}, 0, h', \text{PK.seed}, \mathbf{addr})$
 - 7: $\text{SK} \leftarrow (\text{SK.seed}, \text{SK.prf}, \text{PK.seed}, \text{PK.root})$
 - 8: $\text{PK} \leftarrow (\text{PK.seed}, \text{PK.root})$
 - 9: **return** (SK, PK)
-

6.2 Signature Generation

A Phoenix signature consists of five components: a randomization string R , a 4-byte TFORS counter, a 2-byte TFORS signature-length field, a TFORS signature, and a hypertree signature. The hypertree signature comprises a sequence of d XMSS signatures (one per layer; see Figure 2), each of which includes a GWOTS+C signature, the XMSS authentication path for that layer, and a 4-byte GWOTS+C counter.

Randomness $\mathbf{R}$	n bytes
Counter $\mathbf{ctr}$	4 bytes
TFORS signature $\mathbf{SIG}_{\text{TFORS}}$	$\mathbb{E}[M(k' \cdot t, k)] \cdot n$ bytes
HT signature $\mathbf{SIG}_{\text{HT}}$	$(h + d \cdot l) \cdot n + d \cdot 4$ bytes

Figure 2: Signature data format

SIG _{General WOTS+C}	$len \cdot n$ bytes
AUTH[0]	n bytes
...	n bytes
AUTH[h' - 1]	n bytes
Counter ctr	4 bytes

Figure 3: Signature structure of a single hypertree layer

Algorithm 18 Phoenix_sign_internal

Input: Message M , private key $\mathbf{SK} = (\mathbf{SK.seed}, \mathbf{SK.prf}, \mathbf{PK.seed}, \mathbf{PK.root})$, additional randomness $addrnd$.

Output: Phoenix signature $\mathbf{SIG}$

```

1: addr  $\leftarrow \text{toByte}(0, 32)$ 
2:  $opt\_rand \leftarrow addrnd$ 
3:  $R \leftarrow \mathbf{PRF}_{msg}(\mathbf{SK.prf}, opt\_rand, M)$ 
4:  $ctr \leftarrow 0$ 
5: while true do
6:    $(md, idx_{tree}, idx_{leaf}) \leftarrow \mathbf{H}_{msg}(R, \mathbf{PK.seed}, \mathbf{PK.root}, M \parallel ctr)$ 
7:    $indices \leftarrow \text{message\_to\_indices}(md, \mathbf{PK.seed})$ 
8:    $authCount \leftarrow |\text{Octopus\_Auth\_Indices}(indices, h_{\text{TFORS}})|$ 
9:    $siglen_{\text{TFORS}} \leftarrow (k + authCount) \cdot n$ 
10:  if  $siglen_{\text{TFORS}} \leq \text{TFORS\_SIG\_MAX}$  then
11:    break
12:  end if
13:   $ctr \leftarrow ctr + 1$ 
14: end while
15:  $\mathbf{SIG} \leftarrow R \parallel \text{toByte}(ctr, 4) \parallel \text{toByte}(siglen_{\text{TFORS}}, 2)$ 
16: addr.setTreeAddress( $idx_{tree}$ )
17: addr.setKeyPairAddress( $idx_{leaf}$ )
18:  $(\mathbf{SIG}_{\text{TFORS}}, \mathbf{PK}_{\text{TFORS}}) \leftarrow \text{TFORS\_sign}(indices, \mathbf{SK.seed}, \mathbf{PK.seed}, \mathbf{addr})$ 
19:  $\mathbf{SIG} \leftarrow \mathbf{SIG} \parallel \mathbf{SIG}_{\text{TFORS}}$ 
20:  $\mathbf{SIG}_{\text{HT}} \leftarrow \text{HT\_sign}(\mathbf{PK}_{\text{TFORS}}, \mathbf{SK.seed}, \mathbf{PK.seed}, idx_{tree}, idx_{leaf})$ 
21:  $\mathbf{SIG} \leftarrow \mathbf{SIG} \parallel \mathbf{SIG}_{\text{HT}}$ 
22: return  $\mathbf{SIG}$ 

```

Algorithm 18 describes the internal Phoenix signature generation procedure, in which the message randomizer can operate in either non-deterministic (hedged) or deterministic mode. In the default hedged mode, an n -byte random input is provided as $addrnd$ and used to set opt_rand ; this mode is recommended for platforms where side-channel attacks are a concern. When a random bit gen-

erator is unavailable, the deterministic variant may be used instead: *addrnd* is omitted and *opt_rand* is set to **PK.seed**, producing identical signatures for identical messages. The message digest computed by $\mathbf{H}_{msg}$ is split into three parts: the first $L_{\text{TFORS}} = \max(n, \lceil k \cdot a/8 \rceil)$ bytes are used as the TFORS index *md*, the next $\lceil (h - h')/8 \rceil$ bytes are used as the hypertree tree index *idx_{tree}*, and the last $\lceil h'/8 \rceil$ bytes are used as the XMSS leaf index *idx_{leaf}*.

6.3 Signature Verification

As with signature generation, Phoenix signature verification recomputes the message digest from *R*, the public key, the message, and the transmitted TFORS counter. It then recomputes the TFORS indices, checks that the transmitted TFORS signature length matches the length implied by Octopus pruning, reconstructs the TFORS public key from the TFORS signature, and verifies the hypertree signature against **PK.root**. The full verification procedure is formalized in Algorithm 19.

Algorithm 19 Phoenix_verify_internal

Input: Message *M*, signature **SIG**, public key **PK** = (**PK.seed**, **PK.root**).

Output: Boolean

```

1: addr  $\leftarrow$  toByte(0, 32)
2: R  $\leftarrow$  SIG.getR()
3: ctr  $\leftarrow$  SIG.getCtr()
4: siglenTFORS  $\leftarrow$  SIG.getTFORSLen()
5: SIGTFORS  $\leftarrow$  SIG.getSIG_TFORS()
6: SIGHT  $\leftarrow$  SIG.getSIG_HT()
7: (md, idxtree, idxleaf)  $\leftarrow$   $\mathbf{H}_{msg}(R, \mathbf{PK.seed}, \mathbf{PK.root}, M \parallel ctr)$ 
8: indices  $\leftarrow$  message_to_indices(md, PK.seed)
9: authCount  $\leftarrow$  |Octopus.Auth.Indices(indices, hTFORS)|
10: expectedLenTFORS  $\leftarrow$  (k + authCount) · n
11: if siglenTFORS  $\neq$  expectedLenTFORS then
12:   return REJECT
13: end if
14: addr.setTreeAddress(idxtree)
15: addr.setKeyPairAddress(idxleaf)
16: PKTFORS  $\leftarrow$  TFORS_pkFromSig(PK.seed, addr, md, SIGTFORS)
17: return HT.verify(PKTFORS, SIGHT, PK.seed, idxtree, idxleaf, PK.root)

```

7 Security Proof of Phoenix

The security proof of Phoenix follows the same game-hopping structure as SPHINCS⁺ [4], where each hop is bounded by the security of one of the underlying building

blocks. In our construction, we replace WOTS⁺ [9] with **General WOTS+C** and FORS [4] with **TFORS**. Therefore, to establish the overall security, we provide new security bounds for General WOTS+C under the appropriate notion (d-EU-naCMA), while for TFORS we argue that its security reduces to the interleaved target subset resilience (ITSR) of the message digest function $\mathbf{H}_{msg}$ (see Section 5.3): any adversary that breaks TFORS can be used to break the ITSR property of $\mathbf{H}_{msg}$.

Combining the results of the previous sections with the security framework of SPHINCS⁺ [4], we obtain the following theorem.

Theorem 7.1 (Post-Quantum EU-CMA Security of Phoenix). *For parameters $n, h, d, m, d_{\text{off}}, k, t$ as defined in Sections 4 and 5, let $L_{msg} = \sum_{i=1}^m (2^{s_i} - 1)$ and $L_{\text{off}} = \sum_{j=1}^{d_{\text{off}}} (2^{t_j} - 1)$, where s_i (for $i = 1, \dots, m$) are the Winternitz parameters for the message part and t_j (for $j = 1, \dots, d_{\text{off}}$) are the parameters for the checksum-offset part, as defined in Section 4. Then the following security bound can be obtained for Phoenix:*

$$\begin{aligned} \text{InSec}^{\text{PQ-EU-CMA}}(\text{Phoenix}; \xi, q_s) &\leq \text{InSec}^{\text{PQ-PRF}}(\text{PRF}_{\text{msg}}; \xi, q_s) \\ &\quad + \text{InSec}^{\text{PQ-PRF}}(\text{PRF}; \xi, q_1) \\ &\quad + \text{InSec}^{\text{PQ-ITSR}}(\mathbf{H}_{\text{msg}}; \xi, q_s) \\ &\quad + \text{InSec}^{\text{PQ-SM-TCR}}(\mathbf{T}_\ell; \xi, q_1) \\ &\quad + \text{InSec}^{\text{PQ-SM-TCR}}(\mathbf{H}; \xi, q_2) \\ &\quad + 3 \cdot \text{InSec}^{\text{PQ-SM-TCR}}(\mathbf{F}; \xi, q_3) \\ &\quad + \text{InSec}^{\text{PQ-SM-DSPR}}(\mathbf{F}; \xi, q_3), \end{aligned}$$

where the query bounds are:

$$\begin{aligned} q_1 &< 2^{h+1} \cdot (k' \cdot t + 3), \\ q_2 &< 2^{h+2} \cdot (L_{msg} + L_{\text{off}} + 2k' \cdot t), \\ q_3 &< 2^{h+1} \cdot (L_{msg} + L_{\text{off}} + k' \cdot t), \end{aligned}$$

and q_s denotes the number of signing queries made by the adversary, ξ the overall time resource.

Explanation. The constants 2^{h+1} , 2^{h+2} , etc., account for the number of hash evaluations across all layers of the hypertree; the terms involving $k' \cdot t$, L_{msg} , and L_{off} correspond to the number of leaf nodes and chain lengths in TFORS and General WOTS+C, respectively.

Definitions. - $\text{InSec}^{\text{PQ-ITSR}}(\mathbf{H}_{\text{msg}}; \xi, q_s)$ denotes the maximum success probability of a quantum adversary running in time ξ and making q_s signing queries in breaking the interleaved target subset resilience (ITSR) of $\mathbf{H}_{\text{msg}}$, as defined in Section 5.3. - $\text{InSec}^{\text{PQ-SM-TCR}}(\mathbf{F}; \xi, p)$ and $\text{InSec}^{\text{PQ-SM-DSPR}}(\mathbf{F}; \xi, p)$ denote

the insecurity of a tweakable hash function $\mathbf{F}$ against quantum adversaries for the single-message target collision resistance (SM-TCR) and single-message decisional second-preimage resistance (SM-DSPR) properties, respectively, as defined in Section 2. The hash functions $\mathbf{T}_\ell$, $\mathbf{H}$, and $\mathbf{F}$ appearing in the bound are all instances of tweakable hash functions.

In this section, we analyze the concrete security of Phoenix, which integrates TFORS as the few-time signature component and General WOTS+C as the one-time signature component. Following the security reduction in Theorem 7.1, the overall EU-CMA insecurity is bounded by the sum of the insecurities of its cryptographic building blocks plus the success probability of an interleaved target subset resilience (ITSR) attack against the message digest function $\mathbf{H}_{msg}$.

We first derive the ITSR success probability for a single TFORS instance. Let P_{TFORS} denote this probability:

$$P_{\text{TFORS}} = \mathbb{E}_{(\gamma_1, \dots, \gamma_{k'})} \left[\frac{1}{\binom{k'}{k}} \sum_{S: |S|=k} \prod_{j \in S} \left(1 - \left(1 - \frac{1}{t} \right)^{\gamma_j} \right) \right],$$

where k is the number of secret values selected in TFORS, $t = 2^a$ is the number of elements per set of secret values, k' is the total number of secret value sets in TFORS, and γ_j denotes how many of the q_s observed signatures selected the j -th set.

As shown in Section 5.3, this probability can be upper bounded by:

$$P_{\text{TFORS}} \leq \left[1 - \left(1 - \frac{k}{k't} \right)^{q_s} \right]^k.$$

We now consider the full Phoenix scheme, which organizes 2^h TFORS instances in a hypertree structure. Let P_{ITSR} denote the ITSR success probability for the overall scheme. A forgery attempt targets a specific hypertree leaf (and thus a specific TFORS instance). For the forgery to succeed, the adversary must: 1. Choose a target hypertree leaf (determined by the message digest) 2. Successfully cover all k required secret values in the TFORS instance at that leaf

Let δ be the number of signatures among the q_s queries that targeted the specific TFORS instance where the forgery is attempted. Then $\delta \sim \text{Binomial}(q_s, 1/2^h)$, since each signature independently selects a hypertree leaf uniformly at random. Given that exactly δ signatures hit this leaf, each signature selects k distinct sets from the k' available sets. Thus, the number of times set j is selected among these δ signatures, denoted γ_j , follows $\gamma_j \sim \text{Binomial}(\delta, k/k')$. Note that the γ_j 's are not independent (they must sum to $k\delta$) and exhibit negative dependence due to the without-replacement selection within each signature. However, as shown in Section 5.3, the unconditional expectation over independent Binomial variables provides a valid upper bound due to the negative dependence property.

Let $P_{\text{TFORS}}(\delta)$ denote the ITSR success probability for a TFORS instance given exactly δ signatures:

$$P_{\text{TFORS}}(\delta) = \mathbb{E}_{(\gamma_1, \dots, \gamma_{k'})} \left[\frac{1}{\binom{k'}{k}} \sum_{S: |S|=k} \prod_{j \in S} \left(1 - \left(1 - \frac{1}{t} \right)^{\gamma_j} \right) \right],$$

where $\gamma_j \sim \text{Binomial}(\delta, k/k')$.

Then the probability that the adversary can forge a signature at a specific leaf is:

$$\mathbb{E}_\delta [P_{\text{TFORS}}(\delta)] = \sum_{\delta=1}^{q_s} P_{\text{TFORS}}(\delta) \cdot \binom{q_s}{\delta} \left(\frac{1}{2^h} \right)^\delta \left(1 - \frac{1}{2^h} \right)^{q_s - \delta}.$$

By the union bound over all 2^h hypertree leaves, the total ITSR success probability is:

$$P_{\text{ITSR}} \leq 2^h \cdot \sum_{\delta=1}^{q_s} P_{\text{TFORS}}(\delta) \cdot \binom{q_s}{\delta} \left(\frac{1}{2^h} \right)^\delta \left(1 - \frac{1}{2^h} \right)^{q_s - \delta}.$$

Note that for typical parameter choices where 2^h is large (e.g., $h \geq 60$ for post-quantum security) and $P_{\text{TFORS}}(\delta)$ is small, this union bound remains non-trivial (i.e., less than 1).

Using the upper bound $P_{\text{TFORS}}(\delta) \leq \left[1 - \left(1 - \frac{k}{k't} \right)^\delta \right]^k$ from Section 5.3, we obtain:

$$P_{\text{ITSR}} \leq 2^h \cdot \sum_{\delta=1}^{q_s} \left[1 - \left(1 - \frac{k}{k't} \right)^\delta \right]^k \cdot \binom{q_s}{\delta} \left(\frac{1}{2^h} \right)^\delta \left(1 - \frac{1}{2^h} \right)^{q_s - \delta}.$$

For a classical adversary making q_h hash queries, the total probability is bounded by:

$$(q_h + 1) \cdot P_{\text{ITSR}}.$$

For quantum adversaries, the classical reductions are adapted following the standard SPHINCS⁺ quantum security framework. The query complexity is squared to account for Grover-speeded multi-target attacks; the resulting bounds are consistent with the PQ-prefixed insecurity terms in Theorem 7.1.

Combining this ITSR bound with the other insecurity terms from Theorem 7.1 yields the overall EU-CMA insecurity of Phoenix. This analysis confirms that the scheme achieves the desired security level under the assumptions of post-quantum hardness of the underlying primitives.

8 Parameter Sets for Phoenix

Recall that for each security level, SPHINCS⁺ [4] provides two instantiations or sets of parameters that give a trade-off between faster and smaller signatures. Following this rationale, we also provide two such options for Phoenix: one size-optimized (suffix “s” for “small”) and one speed-optimized (suffix “f” for “fast”) parameter set for each security level. This allows us to select faster parameters when signature speed is prioritized, and to use the other set when signature size is the limiting factor or speed is not a major concern.

For our concrete parameter choices, we searched for parameters that minimize the signature size while maintaining comparable signature speed, security level, and number of supported signatures (2^{64}) as the original variants proposed in the SPHINCS⁺ NIST submission [7].

8.1 Parameter Notation

The following parameters are used to instantiate Phoenix:

- n : Length of private keys, public keys, and hash digests. The security parameter equals to $8n$.
- h : the total height of the hypertree.
- d : the number of layers in the hypertree. The height of each XMSS tree is $h' = h/d$.
- k : the number of secret values selected in a TFORS signature.
- k' : the number of secret value sets in TFORS.
- $t = 2^a$: the number of elements per secret value set in TFORS, where $a = \log_2 t$.

GWOTS+C parameters:

- m : the number of message chains.
- d_{off} : the number of checksum-offset chains.
- $\mathcal{S} = (s_1, \dots, s_m)$: the message profile, where each s_i denotes the bit-width of the i -th message digit, satisfying $\sum_{i=1}^m s_i = 8n - p_{\text{zero}}$, with p_{zero} the number of zero-forced message digits.
- $\mathcal{T} = (t_1, \dots, t_{d_{\text{off}}})$: the offset profile, where each t_j denotes the bit-width of the j -th checksum-offset digit, satisfying $\sum_{j=1}^{d_{\text{off}}} t_j = \lceil \log_2(U - L + 1) \rceil$, with $[L, U]$ the checksum acceptance window.

8.2 Concrete Parameter Choices

Table 4 summarizes the concrete parameters of the SM3-based Phoenix variants. For each variant, Phoenix achieves a smaller signature size than its SPHINCS⁺ counterpart, at the cost of increased signature generation time (see Section ??).

Table 3: Concrete GWOTS+C parameter profiles

Hash	Variant	$\mathcal{S}$	$\mathcal{T}$	m	$m + d_{\text{off}}$	$[L, U]$
SHAKE/SM3 Phoenix-128s	$12 \times 6 + 8 \times 7$	1×5	20	21		[870, 901]
SHAKE/SM3 Phoenix-128f	$20 \times 3 + 17 \times 4$	1×4	37	38		[189, 204]
SHAKE/SM3 Phoenix-192s	$11 \times 6 + 18 \times 7$	1×5	29	30		[1473, 1504]
SHAKE/SM3 Phoenix-192f	$36 \times 3 + 21 \times 4$	1×4	57	58		[275, 290]
SHAKE/SM3 Phoenix-256s	$10 \times 6 + 28 \times 7$	1×5	38	39		[2077, 2108]
SHAKE/SM3 Phoenix-256f	$39 \times 4 + 20 \times 5$	1×4	59	60		[594, 609]
SHAKE/SM3 Phoenix-384s	$8 \times 6 + 48 \times 7$	1×5	56	57		[3284, 3315]
SHAKE/SM3 Phoenix-384f	$66 \times 5 + 9 \times 6$	1×4	75	76		[1298, 1313]
SHAKE/SM3 Phoenix-512s	$94 \times 5 + 7 \times 6$	1×5	101	102		[1661, 1692]
SHAKE/SM3 Phoenix-512f	$4 \times 5 + 82 \times 6$	1×4	86	87		[2637, 2652]

The notation $a \times b$ means that a chains use b -bit message digits. The values are taken from the implementation parameter headers. The interval $[L, U]$ is the accepted checksum window used by the GWOTS+C counter search.

Table 4: Phoenix-SM3 parameter sets

Variant	n	h	d	a	k	k'	Security	Sign size (bytes)
Phoenix-128s	16	70	10	9	14	16	128	6,258
Phoenix-128f	16	68	17	7	19	32	128	13,670
Phoenix-192s	24	63	9	12	19	32	192	13,356
Phoenix-192f	24	68	17	8	29	32	192	30,766
Phoenix-256s	32	66	11	12	24	32	256	24,618
Phoenix-256f	32	64	16	8	47	64	256	44,906
Phoenix-384s	48	66	11	12	37	64	384	54,726
Phoenix-384f	48	68	17	11	39	64	384	88,442
Phoenix-512s	64	64	8	12	57	64	512	98,476
Phoenix-512f	64	68	17	11	58	64	512	138,454

9 Implementation and Performance

9.1 Experimental Setup

We compared the performance of the SM3 version and the SHAKE reference implementation under the same hardware setup in Table 5, and also provided a performance comparison between the SM3 reference implementation and the AVX2 optimized version in Table 6.

The reference implementation of the Phoenix signature algorithm (using SHAKE, no architecture-specific optimizations) was compiled and executed on the same server (Intel Xeon Gold 6226R, 2.90 GHz, Ubuntu 20.04.6, GCC 9.4.0). Taking the parameter set **Phoenix-SHAKE-128f** as an example, the build process consists of navigating to the instance directory `Implementations/Reference_Implementation/Phoenix-SHAKE-128f` and running `make KAT_SIG` to compile the known-answer test executable, followed by `./KAT_SIG` to generate the test vectors. For performance benchmarking, the targets `make benchmark` and `./benchmark` are used. The compiler flags applied during both builds are `-O3 -std=c99 -DPARAMS=phoenix-shake-128f -I. -DALLOW_DEEP_TREES -DSUBMISSION_DRNG -DPARAMNAME="Phoenix-SHAKE-128f"`. The KAT output is written to the `output/` subdirectory and must be byte-identical to the reference test vectors shipped in the submission package. No GPU was employed for any cryptographic operation; all cycle-count measurements were averaged over multiple runs with warm-up iterations, using the `rdtsc` instruction.

The optimized implementation of the Phoenix signature algorithm (using SHAKE or SM3 with AVX2 acceleration, no GPU usage) was compiled and executed on a server equipped with an Intel Xeon Gold 6226R processor (16 physical cores, 32 logical cores, 22 MiB L3 cache, 62 GiB RAM, 2.0 GiB swap). The CPU base frequency is 2.90 GHz with a maximum turbo frequency of 3.90 GHz. The system runs Ubuntu 20.04.6 LTS (Linux kernel 5.15.0-139-generic) and uses GCC 9.4.0 with OpenSSL 1.1.1n. The primary storage is a 953.9 GB NVMe SSD, with an additional 3.7 TB SATA HDD for data. For each parameter set (e.g., **Phoenix-SHAKE-128f-avx2**), the build process consists of navigating to the corresponding subdirectory under `Implementations/Optimized_Implementation/avx2/` and running `make KAT_SIG && ./KAT_SIG` to generate known-answer tests, or `make benchmark && ./benchmark` for performance measurements. The compiler flags used include `-O3 -DNDEBUG -mavx2 -mbmi2 -std=gnu11 -D_POSIX_C_SOURCE=199309L`. Deterministic KAT outputs are compared against the reference test vectors in `Test_Vectors/` and must be byte-identical. Performance benchmarking of the signature primitives (`crypto_sign_keypair`, `crypto_sign`, `crypto_sign_open`) was performed on the same system, pinning the process to a single core, with warm-up iterations and multiple measurement runs averaged; cycle counts were obtained using the `rdtsc` instruction. No GPU was em-

ployed for any cryptographic operation. The AVX2-accelerated implementation exploits SIMD parallelism in the hash functions (SHAKE or SM3) to speed up the hypertree, GWOTS+, and TFORS components. Compared with the reference implementation, the optimized code provides notable speed improvements, the exact factor depending on the security level and the ‘f’ or ‘s’ variant. All signatures generated by the optimized implementation self-verify correctly and are compatible with reference verification.

Measurement methodology. Performance measurements were obtained using the RDTSC instruction (with `constant_tsc`), ensuring accurate cycle counts on single-threaded code. For each parameter set, we ran key generation, randomized signature generation, and verification 100 times, and report the averages over these iterations to ensure statistical significance.

Table 5: Phoenix Reference Performance: SHAKE vs. SM3

Scheme	KeyGen (cycles)	Sign (cycles)	Verify (cycles)	Sign Size (bytes)
Phoenix-SHAKE-128s	405,907,184	3,774,820,890	14,667,520	6,252
Phoenix-SM3-128s	245,571,080	2,477,821,237	8,449,790	6,258
Phoenix-SHAKE-128f	26,310,206	227,762,860	6,179,607	13,668
Phoenix-SM3-128f	19,649,197	164,088,128	3,680,685	13,670
Phoenix-SHAKE-192s	675,069,158	6,228,201,120	22,530,256	13,290
Phoenix-SM3-192s	702,564,317	6,851,044,896	23,704,145	13,356
Phoenix-SHAKE-192f	36,767,297	339,846,306	8,676,365	30,880
Phoenix-SM3-192f	42,177,270	365,448,916	9,402,944	30,766
Phoenix-SHAKE-256s	475,497,580	5,680,432,448	38,383,370	26,624
Phoenix-SM3-256s	508,769,768	6,393,668,360	40,387,335	24,618
Phoenix-SHAKE-256f	65,423,327	629,337,272	17,195,868	47,464
Phoenix-SM3-256f	60,804,558	686,132,265	17,807,532	44,906
Phoenix-SHAKE-384s	737,747,155	9,323,008,682	61,081,260	54,724
Phoenix-SM3-384s	775,262,302	10,140,945,512	64,223,446	54,726
Phoenix-SHAKE-384f	111,582,858	1,562,247,580	40,721,342	88,440
Phoenix-SM3-384f	113,371,569	2,212,502,840	40,544,782	88,442
Phoenix-SHAKE-512s	2,739,473,199	24,976,090,030	44,011,299	98,308
Phoenix-SM3-512s	1,592,795,692	15,319,889,130	25,916,097	98,476
Phoenix-SHAKE-512f	296,362,476	5,899,482,118	139,084,184	121,961
Phoenix-SM3-512f	191,729,180	3,638,891,876	80,390,468	138,454

9.2 Advantages and Limitations of Phoenix

Signature size reduction. Phoenix achieves smaller signatures than SPHINCS⁺ across all comparable parameter sets. For example, Phoenix-SHAKE-128s produces a signature of 6,252 bytes versus 7,856 bytes for SPHINCS⁺-128s, a reduction of approximately 20%. The speed-optimized variant Phoenix-SHAKE-128f

Table 6: Phoenix-SM3 Performance: Reference vs. AVX2 Optimization

Scheme	KeyGen (cycles)	Sign (cycles)	Verify (cycles)	Sign Size (bytes)
Phoenix-SM3-128s (Ref)	245,571,080	2,477,821,237	8,449,790	6,258
Phoenix-SM3-128s-AVX2	123,128,965	1,582,950,245	8,800,616	6,270
Phoenix-SM3-128f (Ref)	19,649,197	164,088,128	3,680,685	13,670
Phoenix-SM3-128f-AVX2	5,146,497	86,221,950	3,794,387	13,690
Phoenix-SM3-192s (Ref)	702,564,317	6,851,044,896	23,704,145	13,356
Phoenix-SM3-192s-AVX2	328,375,398	3,385,507,424	23,355,545	13,226
Phoenix-SM3-192f (Ref)	42,177,270	365,448,916	9,402,944	30,766
Phoenix-SM3-192f-AVX2	9,116,035	192,073,630	9,632,138	30,754
Phoenix-SM3-256s (Ref)	508,769,768	6,393,668,360	40,387,335	24,618
Phoenix-SM3-256s-AVX2	230,909,936	3,294,788,456	40,311,439	24,594
Phoenix-SM3-256f (Ref)	60,804,558	686,132,265	17,807,532	44,906
Phoenix-SM3-256f-AVX2	17,791,296	374,638,490	18,014,556	44,902
Phoenix-SM3-384s (Ref)	775,262,302	10,140,945,512	64,223,446	54,726
Phoenix-SM3-384s-AVX2	700,435,210	9,677,428,358	128,431,753	54,754
Phoenix-SM3-384f (Ref)	113,371,569	2,212,502,840	40,544,782	88,442
Phoenix-SM3-384f-AVX2	71,203,836	2,138,297,776	79,627,495	87,082
Phoenix-SM3-512s (Ref)	1,592,795,692	15,319,889,130	25,916,097	98,476
Phoenix-SM3-512s-AVX2	1,485,584,963	14,162,861,743	51,024,653	98,502
Phoenix-SM3-512f (Ref)	191,729,180	3,638,891,876	80,390,468	138,454
Phoenix-SM3-512f-AVX2	142,507,792	3,644,393,040	158,374,829	138,282

yields 13,668 bytes compared to 17,088 bytes for SPHINCS⁺-128f. This improvement is particularly valuable in bandwidth-constrained or storage-limited environments.

Performance trade-off. The reduction in signature size comes at a moderate increase in signing latency. For the 128-bit small variant, Phoenix signing takes about 6% longer than SPHINCS⁺; for the 128-bit fast variant, the difference is approximately 17%. These overheads remain acceptable for most applications given the gain in signature compactness.

Verification time. Phoenix verification times vary per parameter set. For some configurations (e.g., Phoenix-128f) verification is faster than the SPHINCS⁺ baseline; for others (e.g., Phoenix-256s) it is somewhat slower. This variation arises from the interplay between the TFORS and GWOTS+C parameters and does not affect correctness or security.

Correctness. All tested variants maintained 100% correctness over the 100 randomized signing trials and meet their respective security level claims.

References

1. Abri, M., Katz, J.: Shorter hash-based signatures using forced pruning. Cryptology ePrint Archive, Paper 2025/2069 (2025), <https://eprint.iacr.org/2025/2069>, preprint, not yet peer-reviewed

2. Aumasson, J.P., Endignoux, G.: Improving stateless hash-based signatures. *Cryptology ePrint Archive*, Paper 2017/933 (2017), <https://eprint.iacr.org/2017/933>, preprint
3. Bernstein, D.J., Hopwood, D., Hülsing, A., Lange, T., Niederhagen, R., Papachristodoulou, L., Schneider, M., Schwabe, P., Wilcox-O’Hearn, Z.: SPHINCS: practical stateless hash-based signatures. In: Oswald, E., Fischlin, M. (eds.) *Advances in Cryptology - EUROCRYPT 2015*. *Lecture Notes in Computer Science*, vol. 9056, pp. 368–397. Springer (2015). https://doi.org/10.1007/978-3-662-46800-5_15
4. Bernstein, D.J., Hülsing, A., Kölbl, S., Niederhagen, R., Rijneveld, J., Schwabe, P.: The SPHINCS⁺ signature framework. In: Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds.) *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. pp. 2129–2146. ACM (2019). <https://doi.org/10.1145/3319535.3363229>
5. Buchmann, J., Dahmen, E., Ereth, S., Hülsing, A., Rückert, M.: On the security of the winternitz one-time signature scheme. In: Nitaj, A., Pointcheval, D. (eds.) *Progress in Cryptology - AFRICACRYPT 2011*. *Lecture Notes in Computer Science*, vol. 6737, pp. 363–378. Springer (2011). https://doi.org/10.1007/978-3-642-21969-6_23
6. Buchmann, J., Dahmen, E., Hülsing, A.: XMSS - A practical forward secure signature scheme based on minimal security assumptions. In: Yang, B. (ed.) *Post-Quantum Cryptography - 4th International Workshop, PQCrypto 2011*. *Lecture Notes in Computer Science*, vol. 7071, pp. 117–129. Springer (2011). https://doi.org/10.1007/978-3-642-25405-5_8
7. Cooper, D., et al.: Stateless hash-based digital signature standard (2024)
8. Dods, C., Smart, N.P., Stam, M.: Hash based digital signature schemes. In: Smart, N.P. (ed.) *Cryptography and Coding, 10th IMA International Conference*. *Lecture Notes in Computer Science*, vol. 3796, pp. 96–115. Springer (2005). https://doi.org/10.1007/11586821_8
9. Hülsing, A.: W-OTS⁺ - shorter signatures for hash-based signature schemes. In: Youssef, A.M., Nitaj, A., Hassanien, A.E. (eds.) *Progress in Cryptology - AFRICACRYPT 2013*. *Lecture Notes in Computer Science*, vol. 7918, pp. 173–188. Springer (2013). https://doi.org/10.1007/978-3-642-38553-7_10
10. Kudinov, M., Hülsing, A., Ronen, E., Yogev, E.: SPHINCS⁺C: Compressing SPHINCS⁺ with (almost) no cost. *Cryptology ePrint Archive*, Paper 2022/778 (2022), <https://eprint.iacr.org/2022/778>, preprint
11. Lamport, L.: Constructing digital signatures from a one way function. *Technical Report* (1979)
12. McGrew, D.A., Curcio, M., Fluhrer, S.R.: Leighton-micali hash-based signatures. *RFC 8554*, 1–61 (2019). <https://doi.org/10.17487/RFC8554>
13. Merkle, R.C.: A certified digital signature. In: *Conference on the Theory and Application of Cryptology*. pp. 218–238. Springer (1989)
14. Qin, Z., Sun, S.: Extending the SPHINCS⁺ framework: Varying the tree heights and chain lengths. *Cryptology ePrint Archive*, Paper 2025/2236 (2025), <https://eprint.iacr.org/2025/2236>, preprint

15. Reyzin, L., Reyzin, N.: Better than BiBa: Short one-time signatures with fast signing and verifying. Cryptology ePrint Archive, Paper 2002/014 (2002), <https://eprint.iacr.org/2002/014>, preprint
16. Zhang, K., Cui, H., Yu, Y.: Revisiting the constant-sum winternitz one-time signature with applications to SPHINCS⁺ and XMSS. In: Handschuh, H., Lysyanskaya, A. (eds.) Advances in Cryptology - CRYPTO 2023. Lecture Notes in Computer Science, vol. 14085, pp. 455–483. Springer (2023). https://doi.org/10.1007/978-3-031-38554-4_15

A Address and Member functions

Address Structure. Phoenix uses a 32-byte address to provide domain separation for calls to PRF, **F**, **H**, and **T_ℓ**. The format follows the word-oriented address convention used in SPHINCS⁺/SLH-DSA [7], but extends it with Phoenix-specific address types for GWOTS+C and TFORS.

An address consists of eight 4-byte words encoded in big-endian order. The first words identify the hypertree position, while the type field determines the interpretation of the remaining type-dependent words. Phoenix uses seven address types, with type values from 0 to 7; type value 4 is reserved for future extensions. Whenever the type changes, the type-dependent words are reset before the component-specific fields are set. This convention avoids unintended reuse of address fields across different hashing domains.

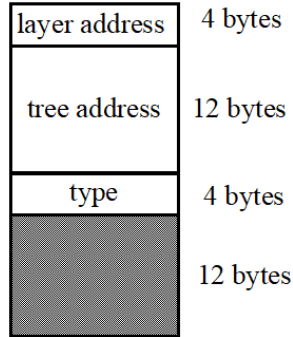


Figure 4: **Address (addr)**

- GWOTS+C `_CHAIN` (*type* = 0) is used for hash-chain computations in GWOTS+C (Figure 5). The type-dependent words encode the GWOTS key-pair address, the chain address, and the hash address within that chain. The key-pair address identifies the GWOTS key pair within the XMSS tree specified by the

layer and tree addresses. The chain address ranges over all GWOTS chains, including both message chains and checksum-offset chains.

- **GWOTSPK** (*type* = 1) is used when compressing the GWOTS chain endpoints into a single GWOTS public key (Figure 6). Its type-dependent words contain the GWOTS key-pair address, while the remaining type-dependent words are set to zero.
- **HASHTREE** (*type* = 2) is used for internal-node computations in XMSS trees (Figure 7). Its type-dependent words are set to (0, *treeHeight*, *treeIndex*), where *treeIndex* identifies the node at height *treeHeight*.
- **TFORSTREE** (*type* = 3) is used for internal-node computations in TFORS trees (Figure 8). Its type-dependent words are set to (*keyPairAddress*, *treeHeight*, *treeIndex*), where *keyPairAddress* identifies the TFORS key pair, and *treeIndex* identifies the node at height *treeHeight*.
- **GWOTSPRF** (*type* = 5) is used to derive the initial secret value of a GWOTS hash chain (Figure 10). The type-dependent words are set to (*keyPairAddress*, *chainAddress*, 0).
- **TFORSPRF** (*type* = 6) is used to derive the secret value associated with a TFORS leaf (Figure 9). For a given leaf, it uses the same key-pair address, tree height, and tree index as the corresponding **TFORSTREE** address, with the type field changed to **TFORSPRF**.
- **COMPRESS_GWOTS** (*type* = 7) is used to domain-separate the counter-search hash in GWOTS+C signing and verification (Figure 11). The type-dependent words are set to (*keyPairAddress*, *counter*, 0), where the counter is varied until the resulting checksum falls inside the admissible window.

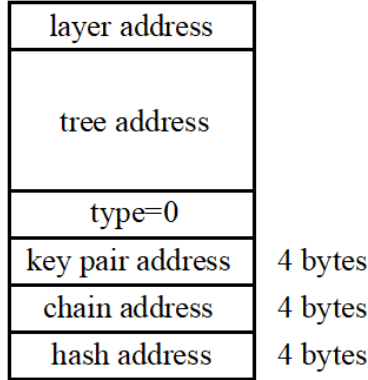


Figure 5: **GWOTS hash address**

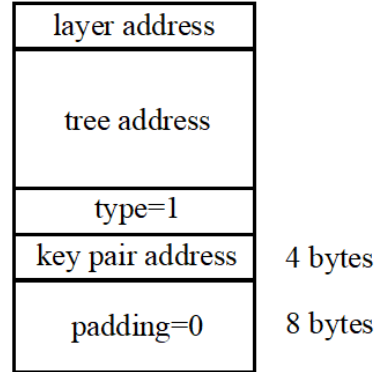


Figure 6: **GWOTS public-key compression address**

layer address	
tree address	
type=2	
padding=0	4 bytes
tree height	4 bytes
tree index	4 bytes

Figure 7: **Hash tree address**

layer address = 0	
tree address	
type=3	
key pair address	4 bytes
tree height	4 bytes
tree index	4 bytes

Figure 8: **TFORS tree address**

layer address = 0	
tree address	
type=6	
key pair address	4 bytes
tree height = 0	4 bytes
tree index	4 bytes

Figure 9: **TFORS key generation address**

Member functions. For the address structure described above, we define a set of member functions that allow for the manipulation of address fields. These functions are summarized in Table 7. Each function takes an address as input and modifies specific fields according to the intended operation. The functions ensure that the address is correctly formatted and that the appropriate fields are set for each use case.

The functions `setChainAddress`, `setTreeHeight`, and `setCounter` write to the same 4-byte word, but they are used under different address types. Similarly, `setHashAddress` and `setTreeIndex` write to the same final word. Therefore the address type determines the interpretation of these fields.

layer address	
tree address	
type=5	
key pair address	4 bytes
chain address	4 bytes
padding=0	4 bytes

Figure 10: **GWOTS key generation address**

layer address	
tree address	
type=7	
key pair address	4 bytes
counter	4 bytes
padding=0	4 bytes

Figure 11: **GWOTS checksum compression address**

B Byte-String and Integer Conversions

This appendix gives the explicit conventions used for byte-string serialization, integer conversion, and fixed-width digest parsing. These routines are deterministic and use big-endian order throughout.

If an array $X = (X[0], \dots, X[m-1])$ consists of m byte strings of length n , its serialized form is

$$X[0] \parallel X[1] \parallel \dots \parallel X[m-1].$$

The function $\text{toInt}(X, n)$ interprets an n -byte string X as the integer

$$\sum_{i=0}^{n-1} X[i] \cdot 256^{n-1-i}.$$

The function $\text{toByte}(x, n)$ returns the n -byte big-endian representation of $x \bmod 256^n$. Unless stated otherwise, all calls in the algorithms satisfy $0 \leq x < 256^n$.

Algorithm 20 $\text{toInt}(X, n)$

Input: n -byte string X .

Output: Integer represented by X in big-endian order.

```

1:  $total \leftarrow 0$ 
2: for  $i$  from 0 to  $n-1$  do
3:    $total \leftarrow 256 \cdot total + X[i]$ 
4: end for
5: return  $total$ 
```

Table 7: Member functions for addresses

Member function	Expanded notation
<code>addr.setLayerAddress(ℓ)</code>	<code>addr $\leftarrow$ toByte(ℓ, 4) $\parallel$ addr[4 : 32]</code>
<code>addr.setTreeAddress(t)</code>	<code>addr $\leftarrow$ addr[0 : 4] $\parallel$ toByte(t, 12) $\parallel$ addr[16 : 32]</code>
<code>addr.setType(Y)</code>	<code>addr $\leftarrow$ addr[0 : 16] $\parallel$ toByte(Y, 4) $\parallel$ addr[20 : 32]</code>
<code>addr.setTypeAndClear(Y)</code>	<code>addr $\leftarrow$ addr[0 : 16] $\parallel$ toByte(Y, 4) $\parallel$ toByte(0, 12)</code>
<code>addr.setKeyPairAddress(i)</code>	<code>addr $\leftarrow$ addr[0 : 20] $\parallel$ toByte(i, 4) $\parallel$ addr[24 : 32]</code>
<code>addr.setChainAddress(i)</code>	<code>addr $\leftarrow$ addr[0 : 24] $\parallel$ toByte(i, 4) $\parallel$ addr[28 : 32]</code>
<code>addr.setTreeHeight(h)</code>	<code>addr $\leftarrow$ addr[0 : 24] $\parallel$ toByte(h, 4) $\parallel$ addr[28 : 32]</code>
<code>addr.setHashAddress(i)</code>	<code>addr $\leftarrow$ addr[0 : 28] $\parallel$ toByte(i, 4)</code>
<code>addr.setTreeIndex(i)</code>	<code>addr $\leftarrow$ addr[0 : 28] $\parallel$ toByte(i, 4)</code>
<code>addr.setCounter(c)</code>	<code>addr $\leftarrow$ addr[0 : 24] $\parallel$ toByte(c, 4) $\parallel$ addr[28 : 32]</code>
<code>$i \leftarrow$ addr.getKeyPairAddress()</code>	<code>$i \leftarrow \text{toInt}(\text{addr}[20 : 24], 4)$</code>
<code>$i \leftarrow$ addr.getTreeIndex()</code>	<code>$i \leftarrow \text{toInt}(\text{addr}[28 : 32], 4)$</code>

Algorithm 21 `toByte( $x, n$ )`

Input: Non-negative integer x , output length n .**Output:** n -byte big-endian encoding of $x \bmod 256^n$.

```

1:  $S \leftarrow 0^n$ 
2:  $total \leftarrow x$ 
3: for  $i$  from 0 to  $n - 1$  do
4:    $S[n - 1 - i] \leftarrow total \bmod 256$ 
5:    $total \leftarrow total \gg 8$ 
6: end for
7: return  $S$ 
```

For GWOTS and TFORS, message digests are parsed into fixed-width digits. Given a bit width b , the function `base2b` maps a byte string to a sequence of integers in $\{0, \dots, 2^b - 1\}$ by reading the digest in big-endian bit order. The value of b is determined by the component that consumes the parsed digest: GWOTS uses the bit width specified by its message profile, whereas TFORS uses the index width specified by its parameter set.

C Comparison with SPHINCS+ and SPHINCS+C

We implemented Phoenix within the official SPHINCS⁺ benchmarking framework [4] to enable a direct, apples-to-apples comparison with the reference

scheme. This section describes our experimental setup and presents performance measurements across all parameter sets.

Table 8: Performance comparison of SPHINCS+, SLH-DSA, SPHINCS+C, and Phoenix-SHAKE

Scenario	KeyGen (cycles)	Sign (cycles)	Verify (cycles)	Sign Size (bytes)
Security Level 128 (s)				
SPHINCS+-128s	510,842,798	3,559,775,735	3,491,640	7,856
SLH-DSA-128s	466,221,060	3,390,998,787	3,243,946	7,856
SPHINCS+C-128s	272,997,676	3,362,910,017	20,682,399	6,304
Phoenix-SHAKE-128s	368,670,206	3,834,482,229	14,568,017	6,252
Security Level 128 (f)				
SPHINCS+-128f	20,364,225	194,395,126	10,241,407	17,088
SLH-DSA-128f	17,828,697	192,434,237	9,042,115	17,088
SPHINCS+C-128f	17,427,368	210,181,261	8,759,368	14,906
Phoenix-SHAKE-128f	11,477,563	218,410,050	6,002,360	13,668
Security Level 192 (s)				
SPHINCS+-192s	724,064,358	6,156,989,607	5,088,923	16,224
SLH-DSA-192s	662,118,383	6,331,167,989	4,726,452	16,224
SPHINCS+C-192s	399,280,063	6,075,936,577	30,985,670	13,776
Phoenix-SHAKE-192s	642,236,178	6,568,112,459	22,877,176	13,290
Security Level 192 (f)				
SPHINCS+-192f	31,807,923	296,837,234	14,758,444	35,664
SLH-DSA-192f	25,422,357	308,696,410	13,411,445	35,664
SPHINCS+C-192f	26,393,445	326,643,679	13,193,038	33,016
Phoenix-SHAKE-192f	16,684,441	331,683,370	8,829,922	30,880
Security Level 256 (s)				
SPHINCS+-256s	493,108,847	5,388,406,691	7,416,560	29,792
SLH-DSA-256s	450,491,509	5,827,854,269	6,917,805	29,792
SPHINCS+C-256s	311,785,782	5,766,562,746	24,715,188	26,096
Phoenix-SHAKE-256s	447,493,768	5,799,414,804	38,834,672	26,624
Security Level 256 (f)				
SPHINCS+-256f	57,199,534	570,365,302	14,980,455	49,856
SLH-DSA-256f	53,578,678	582,762,202	13,578,524	49,856
SPHINCS+C-256f	59,704,327	611,906,940	13,628,532	46,884
Phoenix-SHAKE-256f	34,229,332	665,673,235	17,422,615	47,464